

TanStack Query 5

×

Zustand 5

صفر تا صد

TanStack Query و Zustand

مسیر کامل مدیریت State در اپلیکیشن های React؛ از اولین `useState` تا معماری حرفه ای که در تیم های واقعی شرکت ها استفاده می شود — با زبان فارسی، پروژه های عملی و تمرین های هدفمند.

در این دوره یاد می گیرید داده های سمت کلاینت را با Zustand و داده های سمت سرور را با TanStack Query به شکل حرفه ای مدیریت کنید؛ کش، اینولیدیشن، به روز رسانی خوش بینانه، ذخیره دائمی، تست و الگوهای استاندارد تیمی — همراه با دو پروژه کامل (سبد خرید و اپ بلاگ) که پورتفولیوی شما را متحول می کند.

پیش نیاز: آشنایی مقدماتی با React

سطح: صفر مطلق تا آماده کار

مخاطب: فرانت اند دولوپرها

خروجی: هندل کامل پروژه های واقعی

این کتاب برای کیست و چرا نوشته شده؟

تقریباً هر فرانت‌اند دولوپری در اولین پروژه جدی خود با همین سؤال روبرو می‌شود: «این داده را کجا نگه دارم؟» سید خرید کاربر، وضعیت مودال‌ها، لیست پست‌های گرفته‌شده از سرور، توکن ورود، تنظیمات تم... همه و همه «State» هستند؛ اما همه از یک جنس نیستند و همه در یک ابزار جا نمی‌گیرند.

تجربه سال‌ها پروژه‌های واقعی نشان داده ترکیب Zustand برای State سمت کلاینت و TanStack Query برای State سمت سرور ساده‌ترین، سبک‌ترین و در عین حال استانداردترین معماری در اکوسیستم React است — ترکیبی که امروزه در استارت‌آپ‌ها و شرکت‌های بزرگ به یک انتخاب پیش‌فرض تبدیل شده است. این کتاب دقیقاً همان مسیری را پیش‌می‌گیرد که یک توسعه‌دهنده از صفر تا استخدام در شرکت طی می‌کند.

پیش‌نیازهای لازم

- آشنایی مقدماتی با HTML/CSS و JavaScript مدرن (متغیرها، تابع، آرایه، async/await، destructuring، Object).
- آشنایی مقدماتی با React: کامپوننت، props، JSX و useState. اگر این‌ها را بلد نیستید، یکی دو هفته با مستندات رسمی React پیش بروید و بعد برگردید.
- نصب بودن Node.js نسخه ۲۰ به بالا و یک ویرایشگر مثل VS Code.
- هیچ پیش‌نیازی درباره Redux، MobX یا مدیریت State حرفه‌ای لازم نیست — از صفر شروع می‌کنیم.

ساختار کتاب

بخش چهارم · مرجع سریع (فصل ۱۵) چیت‌شیت دستورها، گلاساری فارسی-انگلیسی و نقشه راه ادامه مسیر — برگه‌ای که همیشه باید دم دست باشد.	بخش سوم · حرفه‌ای (فصل ۱۳ و ۱۴) پروژه ترکیبی کامل با هر دو ابزار به علاوه تست نویسی، دیباگ، آنتی پتن‌ها و آمادگی مصاحبه و استخدام.	بخش دوم · TanStack Query (فصل ۸ تا ۱۲) از تفاوت Server State و Client State تا useQuery، useMutation، مدیریت کش و امکانات پیشرفته مثل Infinite Query.	بخش اول · Zustand (فصل ۱ تا ۷) از مفهوم State و انگیزه ساخت Zustand تا سلکتورها، Middlewareها، الگوهای TypeScript و یک پروژه کامل سید خرید.
---	---	---	---

قراردادهای کتاب

در سراسر کتاب از باکس‌های رنگی زیر استفاده شده تا بتوانید سریع اسکن کنید:

● نکته

تجربه‌ها و ترفندهایی که در پروژه‌های واقعی نجات‌تان می‌دهد.

● هشدار

خطاهای رایجی که اگر ندانید، ساعت ها وقت تان را می گیرند.

● ممنوع

الگوهای اشتباه (آنتی پترن) که در تیم ها بلافاصله رد می شوند.

● تمرین

کارهایی که خودتان باید بنویسید – این کتاب بدون تمرین شما اثری ندارد.

چطور از این کتاب استفاده کنم؟

۱. فصل ها عمداً به ترتیب طراحی شده اند؛ هر فصل روی فصل قبل سوار است. از پرش نترسید، اما بدانید هزینه اش را بعداً می پردازید.

۲. هر کد را خودتان تایپ کنید. کپی-پیست عضله برنامه نویسی را می سوزاند.

۳. تمرین های انتهای فصل ها را حتماً انجام دهید؛ سؤالات مصاحبه معمولاً از دل همین تمرین ها بیرون می آید.

۴. اگر جایی گیر کردید، فصل ۱۵ (چیت شیت و گلاساری) مرجع سریع شماست.

● یادداشت نسخه ها

کدهای این کتاب بر اساس `نسخه ۵ Zustand` و `نسخه ۵ @tanstack/react-query` و `React 18/19` نوشته شده اند. اگر با نسخه ۴ آموزش دیده اید، جدول تفاوت ها در فصل ۸ و ۹ را از دست ندهید.

فهرست مطالب

بخش اول · Zustand — مدیریت State سمت کلاینت

۵	فصل ۱ مدیریت State — چرا، کی و با چه ابزاری؟
۱۱	فصل ۲ Zustand از صفر — نصب و اولین استور
۱۶	فصل ۳ عمق set، get — Zustand و کار با استور از هر جا
۲۱	فصل ۴ Selectors و بهیه سازی رندر
۲۶	فصل ۵ Middlewareها — immer و persist، devtools
۳۲	فصل ۶ الگوهای حرفه‌ای TypeScript — Zustand و اسلایس ها
۳۷	فصل ۷ پروژه عملی ۱ — سبد خرید کامل با Zustand

بخش دوم · TanStack Query — مدیریت Server State

۴۳	فصل ۸ TanStack Query و دنیای Server State
۴۷	فصل ۹ useQuery — خواندن داده از سرور
۵۳	فصل ۱۰ useMutation — تغییر داده و به روزرسانی کش
۵۸	فصل ۱۱ Query Keys و مدیریت کش
۶۳	فصل ۱۲ امکانات پیشرفته — صفحه‌بندی، Infinite و Suspense

بخش سوم · پروژه ترکیبی و آمادگی حرفه‌ای

۶۸	فصل ۱۳ پروژه عملی ۲ — اپ کامل با Zustand و TanStack Query
۷۵	فصل ۱۴ حرفه‌ای شو — تست، دیباگ و آمادگی استخدام

بخش چهارم · مرجع سریع

۸۰	فصل ۱۵ چیت شیت، گلاساری و نقشه راه
----	------------------------------------

مدیریت State – چرا، کی و با چه ابزاری؟

قبل از نصب هر کتابخانه‌ای، باید بدانیم «State» دقیقاً چیست، چند نوع دارد و چرا React به‌تنهایی برای همه آن‌ها کافی نیست. این فصل نقشهٔ کل مسیر دوره است.

در این فصل یاد می‌گیرید

- State چیست و در یک اپلیکیشن واقعی چند نوع State داریم؟
- چرا `useState` و `useContext` در پروژه‌های بزرگ کافی نیستند؟
- تفاوت بنیادین Client State و Server State چیست؟
- چرا ترکیب Zustand و TanStack Query معماری استاندارد امروز است؟

State یعنی چه؟

هر اپلیکیشن React در هر لحظه یک «تصویر از وضعیت» دارد: کاربر کیست؟ سبد خریدش چه چیزی دارد؟ مودالی باز است؟ دادهٔ لیست محصولات هنوز از سرور نیامده؟ به همهٔ این‌ها **State (وضعیت)** می‌گوییم: داده‌هایی که در طول زمان تغییر می‌کنند و رابط کاربری چیزی است که از این داده‌ها رندر می‌شود.

رابطهٔ معروف React را به یاد بیاورید:

$$UI = f(state)$$

رابط کاربری، خروجی یک تابع از State است؛ هر بار State عوض شود، UI دوباره رندر می‌شود.

یعنی کار اصلی ما به‌عنوان توسعه‌دهندهٔ فرانت‌اند، در واقع **مدیریت صحیح State** است: کجا نگهش داریم؟ چه کسی تغییرش بدهد؟ و از کجا بفهمیم که تغییر کرده؟

اما همهٔ State‌ها از یک جنس نیستند

بزرگ‌ترین اشتباه مبتدی‌ها این است که همهٔ داده‌ها را در یک قیف می‌ریزند. یک اپلیکیشن واقعی حداقل چهار نوع State دارد:

نوع State	مثال	ویژگی کلیدی	ابزار مناسب
UI State	باز/بسته بودن مودال، سایدبار، تم روشن/تاریک، تب فعال	کوتاه‌عمر، فقط در همین مرورگر معنا دارد	<code>useState</code> / <code>Zustand</code>
Client Data State	سبد خرید، لیست آهنگ‌های انتخابی، فرم چند مرحله‌ای	داده‌ای که «ما» می‌سازیم و مالکش هستیم	<code>Zustand</code>
Server State	لیست پست‌ها، پروفایل کاربر، موجودی انبار	مالکش سرور است؛ ما فقط نسخه کش‌شده آن را داریم	<code>TanStack Query</code>
URL State	صفحه فعلی، فیلترها، id محصول	باید در آدرس باشد تا لینک‌پذیر و رفرش‌پذیر باشد	React Router

● نکته طلایی

سؤالی که همیشه بپرسید: «مالک حقیقی این داده کیست؟» اگر مالک، خود مرورگر است (سبد خرید، مودال) → Client State است. اگر مالک، دیتابیس سرور است (پست‌ها، موجودی) → Server State است و اگر جایی ذخیره‌اش کنیم فقط یک «کپی محلی و ممکن است کهنه» نگه داشته‌ایم.

چرا `useState` به‌تنهایی کافی نیست؟

`useState` عالی است — برای State محلی یک کامپوننت. مشکل وقتی شروع می‌شود که چند کامپوننت دور از هم به یک State نیاز دارند. رایج‌ترین راه‌حل خام، **Prop Drilling** است: دادن props از کامپوننت ریشه، لابه‌لای کامپوننت‌هایی که اصلاً به آن نیاز ندارند، تا به مقصد برسد.

src/App.jsx — prop drilling

JSX

```
function App() {
  const [cart, setCart] = useState([])
  return <Layout cart={cart} setCart={setCart} />
}

function Layout({ cart, setCart }) {
  // نیاز ندارد؛ فقط جاده است cart اصلاً به Layout
  return <Header cart={cart} setCart={setCart} />
}

function Header({ cart, setCart }) {
  // باز هم فقط جاده
  return <CartBadge cart={cart} setCart={setCart} />
}

function CartBadge({ cart, setCart }) {
  // بالاخره مقصد
  return <span>{cart.length} قلم</span>
}
```

در پروژه‌های واقعی این «جاده‌ها» ده‌ها سطح می‌شوند: تغییر کوچک در State یعنی ویرایش پنج فایل و رندر دوباره کامپوننت‌هایی که ربطی به ماجرا نداشتند.

پس useContext؟ ظاهر فریبنده است

راه حل داخلی Context، React است. یک مقدار را بالا تعریف می‌کنیم و هر کامپوننتی هر جای درخت، مستقیم می‌خواند:

src/CartContext.jsx

JSX

```
const CartContext = createContext(null)

function CartProvider({ children }) {
  const [cart, setCart] = useState([])
  const addItem = (item) => setCart((prev) => [...prev, item])
  return (
    <CartContext.Provider value={{ cart, addItem }}>
      {children}
    </CartContext.Provider>
  )
}

// هر جا:
const { cart, addItem } = useContext(CartContext)
```

برای مقدارهای کم تغییر (مثل کاربر لاگین شده یا تم) این کافی است. اما سه مشکل جدی دارد:

- **رندر همه‌جانبه:** هر تغییری در `value` باعث رندر دوباره همه مصرف‌کننده‌های Context می‌شود — حتی آن‌هایی که به آن قسمت تغییر کرده نیاز ندارند. Context هیچ مکانیزم سلکتور ندارد.
- **Provider جهنمی:** هر Context یک Provider جداگانه می‌خواهد و درخت کامپوننت‌ها زیر لایه‌لایه Providerها دفن می‌شود.
- **بدون اکشن سازمان‌یافته:** منطق تغییر State یا در کامپوننت‌ها پخش می‌شود یا باید در Provider انبار شود؛ در پروژه‌های بزرگ به کد سرگردان تبدیل می‌شود.

● هشدار

Context برای مقادیر کم تغییر طراحی شده (Theme، Locale، کاربر). اگر داده‌ای مدام تغییر می‌کند (سبد خرید، تایمر، موقعیت موس)، Context شما را به آهسته‌ترین اپ جهان تبدیل می‌کند. برای این کار جای درست، یک استور خارجی مثل Zustand است.

معرفی کوتاه: Zustand

Zustand (واژه آلمانی به معنی «وضعیت») کتابخانه‌ای است که توسط **Daishi Kato** — یکی از نویسندگان تیم React/Redux — ساخته شد و امروز به محبوب‌ترین گزینه مدیریت Client State تبدیل شده است. سه ویژگی آن را از بقیه جدا می‌کند:

سلکتور بومی

کامپوننت فقط به تکه‌ای که می‌خواهد گوش می‌دهد؛ بقیه تغییرات رندر نمی‌کند.

حداقل حجم

زیر ۲ کیلوبایت فشرده؛ برای مقایسه Redux Toolkit چندین برابر این است.

بدون Provider

استور یک هوک ساده است؛ هیچ Context و هیچ Wrapper اضافه‌ای دور App نمی‌پیچد.

Zustand همان سید خرید، این بار با

JSX

```
import { create } from 'zustand'

const useCartStore = create((set) => ({
  cart: [],
  addItem: (item) => set((state) => ({ cart: [...state.cart, item] })),
}))

// بدون drilling، هر جا - بدون
const cart = useCartStore((s) => s.cart)
const addItem = useCartStore((s) => s.addItem)
```

شاید الان پیچیده به نظر برسد – نگران نباشید؛ از فصل بعد خط‌به‌خط بازش می‌کنیم.

و معرفی کوتاه: TanStack Query

حالا آن سؤال طلایی: «مالک لیست پست‌ها کیست؟» سرور. پس در React فقط یک کش (Cache) از آن نگه می‌داریم؛ کپی‌ای که ممکن است کهنه شود و باید رفرش شود. مدیریت این کش – گرفتن، نگه‌داشتن، به‌روزرسانی، دوباره‌گیری بعد از خطا – دقیقاً کار TanStack Query است (که با نام قدیمی **React Query** هم می‌شناسند).

بدون آن، همه ما یک الگوی شکننده می‌نویسیم:

«useEffect + fetch» الگوی شکننده

JSX

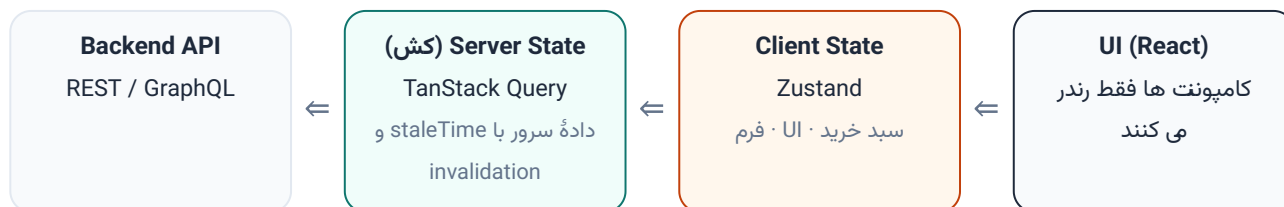
```
const [posts, setPosts] = useState([])
const [loading, setLoading] = useState(true)
const [error, setError] = useState(null)

useEffect(() => {
  setLoading(true)
  fetch('/api/posts')
    .then((r) => r.json())
    .then(setPosts)
    .catch(setError)
    .finally(() => setLoading(false))
}, []) // بدون dedup، بدون retry، بدون کش، بدون refresh...
```

و بعد باید دستی به آن اضافه کنیم: جلوگیری از double-fetch در StrictMode، لغو درخواست هنگام unmount، دوباره‌گیری بعد از برگشتن اینترنت، کش بین تب‌ها، به‌روزرسانی بعد از هر TanStack Query mutation... همه این‌ها را از پیش حل کرده

معماری نهایی دوره

این تصویر ذهنی را تا آخر دوره حفظ کنید – تقریباً هر تصمیم فنی از وی آن پاس می گیرد:



قانون سرانگشتی: هر داده‌ای که از API می‌آید به Zustand نمی‌رود – به کش Query می‌رود. و هر داده‌ای که فقط در مرورگر معنا دارد، به Query نمی‌رود – به Zustand می‌رود. در فصل ۸ کامل این مرز را یاد می‌گیریم؛ فعلاً همین کافی است.

جدول مقایسه: جای ما بین ابزارها کجاست؟

ابزار	نوع	قوت	ضعف برای تیم‌های معمول
<code>useState/Context</code>	داخلی React	بدون کتابخانه	سلکتور ندارد؛ در مقیاس بزرگ رندر اضافه و Provider جهنمی
<code>Redux Toolkit</code>	Client State	بویروپلیت تنظیم‌شده، DevTools عالی	حجم مفاهیم بالا؛ برای اکثر پروژه‌های امروزی بیش‌ازحد سنگین
<code>Zustand</code>	Client State	ساده، سبک، بدون Provider، TS عالی	برای Server State ساخته نشده (و نباید شود!)
<code>Jotai</code>	Client State (اتمیک)	گرانولار بودن عالی	پارادایم اتم‌ها یادگیری دارد؛ برای State دامنه‌ای سنگین‌تر از حد
<code>TanStack Query</code>	Server State	کش، اینولیدیشن، sync بین تب‌ها، retry	جای Client State را نمی‌گیرد (و نباید بگیرد)

● دربارهٔ کد این کتاب

فصل‌های ۱ تا ۷ با JavaScript می‌نویسیم تا مفهوم‌ها سنگین نشوند؛ از فصل ۶ به بعد (و کل بخش TanStack Query) به TypeScript مهاجرت می‌کنیم – همان‌طور که تیم‌های واقعی کار می‌کنند. کد تایپ‌دار در هر فصل، معادل JS همان الگوست.

جمع‌بندی فصل

- State = داده‌ی تغییرپذیری که UI از آن رندر می‌شود؛ $UI = f(state)$.
- چهار نوع State داریم: Server، Client Data، UI و URL – هرکدام ابزار خودش را می‌خواهد.
- Prop Drilling و Context در مقیاس بزرگ می‌شکند؛ Context فقط برای مقادیر کم‌تغییر است.
- Zustand = Client State (سبک خرید، UI). TanStack Query = Server State (هر داده‌ای که مالکش سرور است).
- معماری برنده: $UI \leftarrow Zustand \leftarrow Query\ Cache \leftarrow API$.

● تمرین فصل ۱ – با کاغذ و قلم!

لیست ۱۰ داده از اپی که هر روز استفاده می‌کنید (مثلاً دیجی کالا یا توییتر) بنویسید: سبد خرید، اعلان‌ها، لایک‌ها، تم‌شبانہ و... جلوی هرکدام بنویسید Client State است یا Server State یا URL State و چرا. این تمرین ۱۰ دقیقه‌ای، نیمی از درس فصل ۸ را از قبل در ذهنتان جا می‌اندازد.

در فصل بعد: نصب Zustand و نوشتن اولین استور واقعی – با سه ویژگی که آن را از Context و Redux متمایز می‌کند.

Zustand از صفر – نصب و اولین استور

از نصب تا ساخت اولین استور واقعی: State، اکشن‌ها و مصرف آن‌ها در کامپوننت – بدون حتی یک Provider.

در این فصل یاد می‌گیرید

- نصب Zustand در یک پروژه Vite و ساخت اولین استور
- آناتومی `create`: چرا استور، «هوک» است؟
- نوشتن اکشن‌ها و فراخوانی آن‌ها در UI
- قوانین نام‌گذاری و سازماندهی فایل استورها از روز اول

راه‌اندازی پروژه

برای تمرین‌های بخش اول یک پروژه React ساده با Vite می‌سازیم (اگر پروژه آماده دارید، از همان استفاده کنید):

terminal

BASH

```
npm create vite@latest zustand-lab -- --template react
cd zustand-lab
npm install
npm install zustand      نسخه ۵ را نصب می‌کند
npm run dev
```

● نکته

Zustand به هیچ Wrapper (مثل `QueryClientProvider` یا `Provider` ریداکس) نیاز ندارد. وابستگی واحدش خود React است؛ یعنی حتی می‌توانید آن را در پروژهای غیر React هم استفاده کنید.

اولین استور: شمارنده

کلاسیک‌ترین شروع. یک فایل جدید بسازید: `src/store/useCounterStore.js`

src/store/useCounterStore.js

JS

```
import { create } from 'zustand'

// شروع می‌کنیم use یک «هوک» برمی‌گرداند؛ به همین دلیل اسمش را با create
export const useCounterStore = create((set) => ({
  // ۱) State اولیه
  count: 0,

  // ۲) تغییر می‌دهند State اکشن‌ها: توابعی که
  increment: () => set((state) => ({ count: state.count + 1 })),
  decrement: () => set((state) => ({ count: state.count - 1 })),
}))
```

```
reset: () => set({ count: 0 } ),
}))
```

سه نکته بنیادی در همین ۱۰ خط هست:

- `create(callback)` یک هوک سفارشی بومی گرداند که هر جای برنامه قابل استفاده است – بدون Provider و بدون Context.
- شیء برگشتی callback ترکیبی از داده (اینجا `count`) و اکشن‌ها (اینجا `increment` و...) است. در Zustand هیچ اجباری به جداسازی این دو نیست – حتی توصیه هم می‌شود کنار هم بمانند.
- `set` مثل `setState` در React کار می‌کند: اگر یک شیء جزئی بدهید، فقط همان کلیدها با merge به‌روزرسانی می‌شوند و بقیه State دست‌نخورده می‌ماند.

مصرف در کامپوننت

src/App.jsx

JSX

```
import { useCounterStore } from './store/useCounterStore'

export default function App() {
  // selector را «انتخاب» می‌کنیم مقدار
  const count = useCounterStore((s) => s.count)
  const increment = useCounterStore((s) => s.increment)

  return (
    <main>
      <h1>{count}</h1>
      <button onClick={increment}>+1</button>
    </main>
  )
}
```

کامپوننت با `useCounterStore((s) => s.count)` می‌گوید: «من فقط به `count` کار دارم». نتیجه: اگر بعداً قسمت دیگری از استور تغییر کند، این کامپوننت اصلاً رندر نمی‌شود. (در فصل ۴ به‌عمق این موضوع می‌رویم.)

● هشدار – فراخوانی هوک بدون سلکتور

اگر بنویسید `const state = useCounterStore()` بدون آرگومان، کل استور subscribe می‌شود و هر تغییر، رندر دوباره می‌گیرد. در Zustand نسخه ۴ این شکل مجاز بود؛ از نسخه ۵ استفاده از آن بدون سلکتور فقط برای State ثابت (مثل مقادیر اولیه) مجاز است. عادت کنید: همیشه سلکتور بنویسید.

استور واقعی‌تر: اطلاعات کاربر

یک استور با داده‌های و چند اکشن واقعی، تا merge و تغییر تو در تو را ببینید:

```
import { create } from 'zustand'

const guest = { name: '', email: '', loggedIn: false }

export const useUserStore = create((set) => ({
  user: guest,
  lastAction: null,

  login: (user) => set({ user: { ...user, loggedIn: true }, lastAction: 'login' }),

  logout: () => set({ user: guest, lastAction: 'logout' }),

  // به روزرسانی فقط یک فیلد از شیء تو در تو
  rename: (name) =>
    set((state) => ({
      user: { ...state.user, name },
      lastAction: 'rename',
    })),
}))
```

```
import { useState } from 'react'
import { useUserStore } from './store/useUserStore'

export default function Profile() {
  const user = useUserStore((s) => s.user)
  const login = useUserStore((s) => s.login)
  const rename = useUserStore((s) => s.rename)

  const [name, setName] = useState('')

  if (!user.loggedIn) {
    return <button onClick={() => login({ name: 'مهدی', email: 'm@example.com' })}>ورود</button>
  }

  return (
    <div>
      <p>سلام {user.name}! ({user.email})</p>
      <input value={name} onChange={(e) => setName(e.target.value)} placeholder="نام جدید" />
      <button onClick={() => rename(name)}>تغییر نام</button>
    </div>
  )
}
```

دقت کنید: `rename` شیء `user` را با `spread` بازسازی می کند و فقط `name` را عوض می کند. Zustand مثل React به **immutability** پایبند است؛ هر تغییر یعنی ساخت نسخه جدید از همان قسمت. (فصل ۵ نشان می دهد چطور با `immer`،

این نوشتن را ساده کنیم.)

set – دو صورت کاربرد

صورت	مثال	کی استفاده کنیم؟
شیء جزئی (Partial)	<code>set({ count: 0 })</code>	وقتی مقدار جدید به State فعلی وابسته نیست
تابع آپدیتور	<code>set((s) => ({ count: s.count + 1 })))</code>	وقتی به State فعلی نیاز دارید – همیشه امن تر است چون اگر دو آپدیت پشت سرهم برسند، از آخرین نسخه State حساب می کند

● ممنوع – جهش (Mutation) مستقیم

یعنی جهش State موجود؛ React متوجه تغییر نمی شود، مقایسه مرجع `set((s) => { s.count++ ; return s })` خراب می شود و بگ های «رندر نمی شود» تولید می کند. همیشه یک شیء جدید برگردانید. (فقط در فصل ۵ با میان افزار immer قانونی می شود.)

سازماندهی فایل ها از روز اول

یک قرارداد ساده که در تیم ها جواب می دهد: برای هر دامنه مستقل، یک فایل استور با پیشوند `use`:

ساختار پیشنهادی

TEXT

```
src/
├── store/
│   ├── useCounterStore.js
│   ├── useUserStore.js
│   └── useCartStore.js    # فصل ۷ همین را کامل می کنیم
├── components/
└── App.jsx
```

- اسم استور = `use` + نام دامنه + `Store` (مثلاً `useCartStore`).
- هر استور فقط مسئول دامنه خودش است؛ «استور مادر همه چیز» نسازید – همین باعث فساد State می شود.
- اکشن ها را با فعل نام گذاری کنید: `addItem`، `clearCart`، `toggleTheme`.

جمع‌بندی فصل

- یک هوک بومی گرداند؛ بدون Provider در هر کامپوننتی مصرف می‌شود.
- استور = داده اولیه + اکشن‌ها، کنار هم؛ اکشن‌ها با فعل نام‌گذاری می‌شوند.
- در کامپوننت همیشه با سلکتور بخوانید: `useXStore((s) => s.x)`.
- `set` دو صورت دارد: شیء جزئی یا تابع آپدیت؛ برای State وابسته، تابع آپدیت.
- هرگز State را جهش ندهید؛ نسخه جدید بسازید.

● تمرین فصل ۲

یک استور `useThemeStore` بسازید با `theme: 'light' | 'dark'` State و اکشن `toggleTheme`. در اپ یک دکمه بگذارید که تم را عوض می‌کند و رنگ پس‌زمینه صفحه را بر اساس State عوض کنید. بعد یک استور `useTodoStore` با آرایه `todos` و سه اکشن `removeTodo`، `addTodo` و `toggleTodo` بنویسید و یک لیست کار ساده بسازید — بدون هیچ prop drilling.

در فصل بعد: عمق API استور — پارامتر `get`، اکشن‌های `async` و خواندن/نوشتن State از بیرون دنیای React (در `interceptor`ها و توابع کمکی).

عمق set، get و کار با استور از هر جا

قدرت واقعی Zustand وقتی آشکار می‌شود که بفهمید استور فقط یک شیء زنده خارج از React است: از اکشن‌های async تا خواندن و نوشتن State داخل توابع معمولی جاوااسکریپت.

در این فصل یاد می‌گیرید

- پارامتر `get` در اکشن‌ها و حالت `replace` در `set`
- نوشتن اکشن‌های async با الگوی `loading/error`
- API کامل استور: `subscribe`، `setState`، `getState` – بیرون از React
- الگوی `reset` و استخراج State اولیه

پارامتر `get`: خواندن آخرین نسخه State داخل اکشن

تا حالا همه اکشن‌ها فقط `set` را دیسکت می‌کردند. اما `create` دو پارامتر به شما می‌دهد: `get`، `(set, get)`. همان لحظه، آخرین نسخه State را بوی گرداند:

src/store/useWishlistStore.js

JS

```
import { create } from 'zustand'

export const useWishlistStore = create((set, get) => ({
  items: [],

  // اول بخوان، بعد تصمیم بگیر با get یا toggle
  toggle: (product) => {
    const exists = get().items.some(i => i.id === product.id)
    if (exists) {
      set({ items: get().items.filter(i => i.id !== product.id) })
    } else {
      set({ items: [...get().items, product] })
    }
  },

  clear: () => set({ items: [] }),
}))
```

همین کار را می‌شد داخل `set((state) => ...)` هم کرد؛ تفاوت ظریف اینجاست: اگر اکشن چند `set` پشت‌سرهم داشته باشد، `get()` همیشه نسخه تازه را می‌دهد اما پارامتر `state` داخل یک `set` فقط نسخه همان لحظه است. در اکشن‌های چندمرحله‌ای (مخصوصاً async) `get` انتخاب مطمئن‌تری است.

حالت کم‌کاربرد اما مهم: `replace`

`set` یک آرگومان دوم دارد. اگر `true` بدهید، به جای `merge`، کل State جایگزین می‌کند:

اکشن‌ها هم حذف می‌شوند! تقریباً هرگز این کار را نکنید `set({ items: [] }, true)` //

اولیه — فصل ادامه دارد State ریست کامل به `replace`: کاربرد درست //

● هشدار

کل استور (از جمله اکشن‌ها) را با آن شیء عوض می‌کند. اگر اکشن‌ها را در شیء جدید نیاورده باشید، عملاً استور را می‌سوزانید. ۹۹٪ موارد باید `replace`زننید.

اکشن‌های `async`: بارگذاری داده با `loading` و `error`

اکشن‌ها توابع معمولی هستند؛ پس می‌توانند `async` باشند. الگوی استاندارد برای فراخوانی API داخل استور:

src/store/useNotificationsStore.js

JS

```
import { create } from 'zustand'

export const useNotificationsStore = create((set, get) => ({
  items: [],
  status: 'idle', // idle | loading | success | error
  error: null,

  fetchAll: async () => {
    if (get().status === 'loading') return // جلوگیری از درخواست موازی
    set({ status: 'loading', error: null })
    try {
      const res = await fetch('/api/notifications')
      if (!res.ok) throw new Error(`HTTP ${res.status}`)
      const items = await res.json()
      set({ items, status: 'success' })
    } catch (err) {
      set({ status: 'error', error: err.message })
    }
  },

  markAllRead: async () => {
    const ids = get().items.filter(n => !n.read).map(n => n.id)
    await fetch('/api/notifications/read', {
      method: 'POST',
      body: JSON.stringify({ ids }),
    })
    set((s) => ({
      items: s.items.map(n => ({ ...n, read: true })),
    }))
  },
}))
```

```
import { useEffect } from 'react'
import { useNotificationsStore } from '../store/useNotificationsStore'

export default function Notifications() {
  const items = useNotificationsStore((s) => s.items)
  const status = useNotificationsStore((s) => s.status)
  const fetchAll = useNotificationsStore((s) => s.fetchAll)

  useEffect(() => {
    fetchAll()
  }, [fetchAll])

  if (status === 'loading') return <p>در حال بارگذاری</p>
  if (status === 'error') return <p>خطا در دریافت اعلانها</p>
  return <ul>{items.map((n) => <li key={n.id}>{n.text}</li>)}</ul>
}
```

● هشدار – این کد را در پروژه واقعی برای داده سرور ننویسید!

الگوی بالا برای «فراخوانی های دستورالعملی» (مثلاً ثبت لاگ، ارسال فرم به API خارج از کش) مناسب است، اما برای داده‌ای که از سرور می‌خوانید و در UI نشان می‌دهید، نوشتن دستی loading/error/caching یعنی بازاختراع TanStack Query به شکل شکسته. از فصل ۸ به بعد، خواندن داده فقط با Query انجام می‌شود و این الگو را در جای درستش می‌بینیم.

استور، فراتر از کامپوننت: getState و setState

هوکي که `create` بوی گرداند، خودش چند متد اضافه دارد که استور را از «دنیای بیرون React» قابل استفاده می‌کند:

متد	کار	زدر می‌کند؟
<code>useXStore.getState()</code>	خواندن لحظه‌ای State (بدون subscribe)	خیر
<code>useXStore.setState(partial)</code>	نوشتن مستقیم State از بیرون اکشن‌ها	بله (برای مشترکین)
<code>useXStore.subscribe(fn)</code>	گوش دادن به هر تغییر (بازگشت unsubscribe)	خیر
<code>useXStore.destroy()</code>	پاک‌سازی استور (به‌ندرت، در تست‌ها)	—

کاربردهای واقعی‌ای که در پروژه‌های شرکت‌ها هر روز می‌بینید:

```
import { useAuthStore } from '../store/useAuthStore'

// جایی که هوک نداریم – interceptor خواندن توکن در (۱)
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

```

})

api.interceptors.response.use(
  (res) => res,
  (err) => {
    // نوشتن مستقیم از بیرون: 401 یعنی نشست منقضی شده (۲)
    if (err.response?.status === 401) {
      useAuthStore.getState().logout()
    }
    return Promise.reject(err)
  }
)

```

گوش دادن به تغییرات — بدون رندر

JS

```

// یک بار برای همیشه — داخل App.jsx
useEffect(() => {
  const unsubscribe = useCartStore.subscribe((state) => {
    // با سرور، هشدار سید پر sync، مثال: گزارش گیری
    if (state.items.length > 20) console.warn('سبد خیلی پر شد')
  })
  return unsubscribe // پاک سازی هنگام unmount
}, [])

```

● نکته

«event handler در getState» یکی از رایج ترین الگوهای بهینه سازی است: وقتی در `onClick` به مقدار فعلی نیاز دارید ولی نمی خواهید تغییراتش باعث رندر شود، به جای `subscribe` به آن در کامپوننت، همان لحظه `getState()` بخوانید. (کاملش در فصل ۴ زیر عنوان «Transient Updates».)

الگوی reset: استخراج State اولیه

با رشد استور، «ریست به اول» سخت می شود. راه استاندارد، بیرون کشیدن State اولیه در یک ثابت است:

src/store/useFiltersStore.js

JS

```

import { create } from 'zustand'

const initialState = {
  query: '',
  category: 'all',
  minPrice: 0,
  sort: 'newest',
}

export const useFiltersStore = create((set) => ({
  ...initialState,
  setQuery: (query) => set({ query }),
  setCategory: (category) => set({ category }),

```

```

setSort: (sort) => set({ sort }),
reset: () => set({ ...initialState }), // replace شیء تازه؛ نه
}))

```

دو نکته: اول اینکه `reset` با `merge` ساده انجام شد (بدون `true`)، چون اکشن‌ها جزء State هستند و باید بمانند. دوم اینکه استور فیلترها نمونه خوبی از مرز Client State و URL State است – در پروژه فصل ۱۳ بحثش می‌کنیم.

جمع‌بندی فصل

- `get()` آخرین نسخه State را داخل اکشن می‌دهد؛ برای اکشن‌های چندمرحله‌ای و `async`.
- `set(partial, true)` کل State را جایگزین می‌کند – تقریباً هرگز لازم نیست.
- اکشن `async` با الگوی `idle → loading → success/error` نوشته می‌شود؛ اما برای «داده سرور» جای این الگو TanStack Query است.
- `getState` / `setState` / `subscribe` استور را به لایه‌های غیر React (`interceptor`، وتر، توابع کمکی) وصل می‌کنند.
- ریست استاندارد = استخراج `initialState` + اکشن `reset` با `merge` ساده.

● تمرین فصل ۳

استور `useAuthStore` بسازید با فیلدهای `token` و `user` و اکشن‌های `login(email, password)` با `async` با `delay` ساختگی ۵۰۰ میلی ثانیه و الگوی `loading`، `logout` و `reset`. بعد یک تابع `api` ساختگی بنویسید که با `getState()` توکن را بخواند و اگر توکن نبود، `Promise reject` کند. خروجی را در کنسول با `subscribe` دنبال کنید و ببینید هر اکشن چه تغییری می‌سازد.

Selectors و بهینه‌سازی رندر

سلکتور قلب کارایی Zustand است. در این فصل یاد می‌گیرید دقیقاً چه چیزی باعث رندر می‌شود، چرا برگرداندن شیء جدید از سلکتور خطرناک است و با useShallow و Transient Updates چطور حرفه‌ای بنویسیم.

در این فصل یاد می‌گیرید

- مکانیزم subscribe و مقایسه `Object.is`
- چرا `(s) ≥ ({ a: s.a, b: s.b })` باگ رندر می‌سازد؟
- استفاده صحیح از `useShallow`
- Transient Updates: خواندن و اجرا بدون رندر

چه چیزی باعث رندر می‌شود؟

وقتی می‌نویسید:

مکانیزم سلکتور

JS

```
const price = useProductStore((s) => s.price)
```

پشت صحنه این اتفاق می‌افتد: کامپوننت در استور **subscribe** می‌شود. هر بار استور تغییری می‌کند، Zustand سلکتور شما را با State جدید دوباره اجرا می‌کند و نتیجه را با نتیجه قبلی مقایسه می‌کند — با `Object.is`. اگر برابر باشند، رندری در کار نیست. اگر متفاوت باشند، کامپوننت رندر می‌شود.

پس قانون کارایی Zustand یک جمله است: سلکتور شما باید برای داده تغییرنکرده، همیشه «همان مقدار» (برابر از نظر `Object.is`) را برگرداند.

مقادیر اولیه امن‌اند؛ مقادیر مرجعی خطر

سلکتور	نوع خروجی	هر تغییر استور باعث رندر؟
<code>(s) => s.count</code>	عدد/رشته/بولین	فقط وقتی خودش عوض شود — امن
<code>(s) => s.user.name</code>	رشته	فقط وقتی name عوض شود — امن
<code>(s) => s.items</code>	مرجع آرایه	فقط وقتی مرجع آرایه عوض شود — امن
<code>(s) => s.items.map(...)</code>	آرایه جدید	همیشه! — باگ رندر
<code>(s) => ({ a: s.a, b: s.b })</code>	شیء جدید	همیشه! — باگ رندر

مقایسه `Object.is` برای اعداد و رشته‌ها «ارزشی» است، اما برای آرایه و شیء «مرجعی»: هر اجرای `s.items.map(...)` یک آرایه جدید در حافظه می‌سازد که هرگز با قبلی برابر نیست؛ پس هر تغییر استور (حتی تغییر یک فیلد بی‌ربط) رندری می‌گیرد.

بدتر: چون خود State هم هر بار عوض می‌شود، در برخی سناریوها حلقه بی‌نهایت رندر می‌سازد.

راه حل ۱: سلکتورهای کوچک و متعدد

به جای یک سلکتور شیء‌ساز، چند سلکتور مجزا:

مستقل subscribe خوب — دو

JSX

```
const name = useUserStore((s) => s.user.name)
const email = useUserStore((s) =>
s.user.email)
```

بد — شیء جدید در هر رندر

JSX

```
const { name, email } = useUserStore(
(s) => ({ name: s.user.name, email:
s.user.email })
)
```

فراتر از درستی: این شکل هم دقیق‌تر است (فقط تغییر name زدر می‌گیرد، نه تغییر email) هم خواناتر.

راه حل ۲: useShallow — مقایسه سطحی

گاهی واقعاً چند فیلد را یک‌جا می‌خواهید (مثلاً در یک map یا بازگشت از یک تابع کمکی). برای این موارد Zustand یک هوک کمکی دارد که خروجی سلکتور را سطحی مقایسه می‌کند: کلیدها و مقادیر اولیه برابر = برابر.

و کامپوننت src/store/useCartStore.js

JSX

```
import { useShallow } from 'zustand/react/shallow'
import { useCartStore } from './useCartStore'

function CartSummary() {
  const { items, coupon } = useCartStore(
    useShallow((s) => ({ items: s.items, coupon: s.coupon })))
  // عوض شود رندر می‌گیرد coupon یا مقدار items فقط وقتی مرجع
  return <span>{items.length} کوبن — ظم: {coupon}</span>
}
```

● هشدار — shallow یعنی سطحی، نه عمیق

`useShallow` فقط یک سطح مقایسه می‌کند. اگر سلکتور هر بار محتوای جدیدی بسازد (مثل `s.items.map(i => i.id * 2)`) که عناصرش آرایه/شیء تازه‌اند، باز هم هر بار متفاوت است. برای داده محاسبه‌شده سنگین، ره حل درست `useMemo` در کامپوننت یا سلکتور ذخیره‌شده در استور است.

● نکته — تفاوت با نسخه ۴

در نسخه ۴ آرگومان دوم هوک را برابر `shallow` می‌گذاشتند: `useStore(selector, shallow)`. این API در نسخه ۵ حذف شده و به جایش `useShallow` دور سلکتور پیچیده می‌شود. در مصاحبه‌ها هم می‌پرسند!

محاسبات مشتق شده: کجا انجام شود؟

مجموع قیمت سبد از داده خام مشتق می شود. سه جای ممکن:

- در استور، هنگام هر **mutation** (نگه داشتن **total** در State): سریع ولی تکرار اطلاعات و ریسک ناهماهنگی – معمولاً نه.
- در سلکتور: **(s) ≥ s.items.reduce(...)** – هر اجرا عدد جدیدی «ارزشی» می سازد پس رندر مشکل شیء را ندارد؛ اما محاسبه در هر تغییر استور اجرا می شود.
- در کامپوننت با **useMemo**: فقط وقتی مرجع **items** عوض شود محاسبه می شود – انتخاب پیش فرض برای محاسبات سنگین.

src/CartTotal.jsx

JSX

```
import { useMemo } from 'react'
import { useCartStore } from './store/useCartStore'

export default function CartTotal() {
  const items = useCartStore((s) => s.items)

  // عوض شود دوباره حساب می شود items فقط وقتی آرایه
  const total = useMemo(
    () => items.reduce((sum, i) => sum + i.price * i.qty, 0),
    [items]
  )

  return <div>تومان {total.toLocaleString('fa-IR')} جمع کل</div>
}
```

Transient Updates: بدون رندر با State کار کن

گاهی State تغییری می کند که اصلاً نباید UI را رندر کند: موقعیت موس، مقدار اسکرول، تایمر... برای این ها subscribe و getState بدون هوک استفاده می شود:

src/MousePad.jsx – رندر صفر در هر حرکت

JSX

```
import { useRef, useEffect } from 'react'
import { useMouseStore } from './store/useMouseStore'

export default function MousePad() {
  const ref = useRef(null)

  useEffect(() => {
    // React در setState نوشتن: هر حرکت موس را در استور ذخیره کن - بدون ۱)
    const onMove = (e) => useMouseStore.setState({ x: e.clientX, y: e.clientY })
    window.addEventListener('mousemove', onMove)

    // React را مستقیم آپدیت کن، نه با رندر DOM: خواندن ۲)
    const unsubscribe = useMouseStore.subscribe((s) => {
```

```

    if (ref.current) {
      ref.current.style.transform = `translate(${s.x}px, ${s.y}px)`
    }
  })

  return () => {
    window.removeEventListener('mousemove', onMove)
    unsubscribe()
  }
}, [])

return <div ref={ref} className="cursor-dot" />
}

```

و اگر فقط در یک رویداد به مقدار لحظه‌ای نیاز دارید، اصلاً subscribe لازم نیست:

خواندن در لحظه نیاز

JS

```

const handleSave = () => {
  const { draft } = useEditorStore.getState() // بدون رندر، بدون سلکتور
  saveToServer(draft)
}

```

چک‌لیست کارایی سلکتورها

- ✓ پیش‌فرض: سلکتور مقدار اولیه تکی — `s.a.b` $\geq$ `(s)`.
- ✓ هیچ‌وقت شیء/آرایه تازه داخل سلکتور نسازید.
- ✓ چند فیلد؟ یا چند سلکتور، یا `useShallow` — نه `object literal` خام.
- ✓ محاسبه سنگین؟ `useMemo` در کامپوننت روی مرجع پایدار.
- ✓ State پرتغییر و بی‌نیاز از رندر؟ Transient pattern با `setState/subscribe`.
- ✓ برای راستی‌آزمایی، از «Profiler» tab در React DevTools استفاده کنید و تعداد رندها را ببینید.

جمع‌بندی فصل

- رندر وقتی اتفاق می‌افتد که خروجی سلکتور با `Object.is` تغییر کرده باشد.
- شیء/آرایه تازه در سلکتور = رندر اضافه = باگ کلاسیک Zustand.
- `useShallow` از `zustand/react/shallow` مقایسه سطحی می‌دهد؛ در ۷۵ جایگزین آرگومان دوم هوک است.
- محاسبات مشتق‌شده را با `useMemo` در کامپوننت انجام دهید.
- Transient Updates برای داده پرتغییر بی‌نیاز از رندر: `getState` و `setState` مستقیم.

● تمرین فصل ۴

در پروژه تمرینی یک استور بسازید که هر ۱۰۰ میلی ثانیه یک شمارنده را افزایش دهد (با `setInterval` و `setState`). دو کامپوننت بسازید: یکی با سلکتور معمولی مقدار شمارنده (که رندر می‌شود) و یکی که با Transient pattern فقط عدد را داخل `ref` آپدیت می‌کند. با React DevTools Profiler تفاوت تعداد رندها را ثبت کنید و در یک پاراگراف توضیح دهید هر کدام کجا مناسب است.



Middlewareها — persist، immer و devtools

میان‌افزارها رفتار استور را در چند خط upgrade می‌کنند: ذخیره دائمی در localStorage، دیباگ زمان‌سفر با Redux DevTools و نوشتن کد قابل جهش با immer.

در این فصل یاد می‌گیرید

- مفهوم middleware و ترتیب صحیح ترکیب آن‌ها
- `persist`: ذخیره دائمی با `version`، `partialize` و `migrate`
- `devtools`: دیدن اکشن‌ها در Redux DevTools
- `immer`: نوشتن State به شکل قابل جهش، امن
- ساخت یک middleware سفارشی ساده

Middleware چیست؟

یک **middleware** (میان‌افزار) تابعی است که دور `create` پیچیده می‌شود و رفتار استور را تغییر می‌دهد — بدون اینکه منطق شما را عوض کند. حالت کلی:

قالب عمومی

JS

```
import { create } from 'zustand'
import { someMiddleware } from 'zustand/middleware'

const useStore = create(
  someMiddleware((set, get) => ({
    // استور عادی شما ...
  })))
)
```

می‌توان چند middleware را تو در تو ترکیب کرد. ترتیب توصیه‌شده و رایج (از بیرون به داخل): `persist` ← `devtools` ← بقیه (مثل `immer`):

ترکیب استاندارد سه‌گانه

JS

```
import { create } from 'zustand'
import { persist, devtools } from 'zustand/middleware'
import { immer } from 'zustand/middleware/immer'

const useStore = create(
  devtools(
    persist(
      immer((set) => ({
        items: [],
        addItem: (item) => set((state) => {
```

```

    state.items.push(item) // مجاز است با immer
  }},
  })),
  { name: 'cart-storage' }
)
)
)
)

```

persist – ذخیره دائمی State

پیش‌فرض Zustand در حافظه است: رفرش صفحه یعنی خداحافظی با سبد خرید. میان‌افزار **persist** هر تغییر State را در **localStorage** ذخیره و هنگام اجرا بومی گرداند:

src/store/useCartStore.js

JS

```

import { create } from 'zustand'
import { persist, createJSONStorage } from 'zustand/middleware'

export const useCartStore = create(
  persist(
    (set, get) => ({
      items: [],
      coupon: null,
      _hasHydrated: false,

      addItem: (product) => {
        set((s) => {
          const found = s.items.find((i) => i.id === product.id)
          if (found) {
            return {
              items: s.items.map((i) =>
                i.id === product.id ? { ...i, qty: i.qty + 1 } : i
              ),
            }
          }
          return { items: [...s.items, { ...product, qty: 1 }] }
        })
      },

      removeItem: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),
      clearCart: () => set({ items: [], coupon: null }),

      setHasHydrated: (v) => set({ _hasHydrated: v }),
    }),
    {
      name: 'cart-storage', // اجباری localStorage کلید در
      storage: createJSONStorage(() => localStorage), // پیش‌فرض همین است؛ برای وضوح نوشتیم
      partialize: (state) => ({ // فقط داده ذخیره شود، نه اکشن‌ها
        items: state.items,
        coupon: state.coupon,
      }),
      onRehydrateStorage: () => (state) => {
        state?.setHasHydrated(true) // پایان بازبازی
      }
    }
  )
)

```

```

    },
  }
)
)

```

چهار آپشن مهم persist:

کار	آپشن
کلید ذخیره‌سازی (اجباری). یکتا و ثابت نگهش دارید.	<code>name</code>
محل ذخیره: <code>sessionStorage</code> ، <code>localStorage</code> یا هر Storage سازگار. پیش‌فرض <code>localStorage</code> است.	<code>storage</code>
انتخاب بخشی از State برای ذخیره – اکشن‌ها را ذخیره نکنید (بلااستهلاک و بزرگ می‌شوند).	<code>partialize</code>
کنترل نسخه ساختار ذخیره‌شده و مهاجرت داده‌های قدیمی.	<code>version</code> + <code>migrate</code>
هوک پایان بازیابی – برای ست کردن فلگ hydration یا لاگ.	<code>onRehydrateStorage</code>

version و migrate – وقتی شکل داده عوض می‌شود

فرض کنید بعد از انتشار محصول، فیلد `qty` را به آیتم‌ها اضافه کرده‌اید؛ کاربرانی که نسخه قبلی را در `localStorage` دارند، داده بی‌شکل می‌گیرند. راه‌حل:

مهاجرت داده قدیمی

JS

```

persist(storeApi, {
  name: 'cart-storage',
  version: 2, // نسخه فعلی
  migrate: (persisted, version) => {
    if (version === 1) {
      // نداشت → برابر ۱ قرار بده qty نسخه ۱: هر آیتم
      return {
        ...persisted,
        items: persisted.items.map((i) => ({ ...i, qty: i.qty ?? 1 })),
      }
    }
    return persisted
  },
})

```

مشکل فلش داده: جلوی رندر «سبد خالی» را بگیرید

بازیابی از `localStorage` غیرهمزمان است (در v5 با یک میکروتسک). برای یک فریم، State اولیه (سبد خالی) رندر می‌شود و بعد داده می‌آید – کاربر لحظه‌ای «سبد خالی» می‌بیند و اگر بعدش چیزی اضافه کند، ممکن است روی داده hydrate نشده بنویسد. الگوی حرفه‌ای: فلگ `_hasHydrated` (که در کد بالا دیدید) و تا `true` نشده، الی پایدار نشان ندهید:

src/CartPage.jsx

JSX

```
function CartPage() {
  const hasHydrated = useCartStore((s) => s._hasHydrated)
  const items = useCartStore((s) => s.items)

  if (!hasHydrated) return <p>در حال بازیابی سبد</p>
  if (items.length === 0) return <p>سبد خرید خالی است</p>
  return <CartList />
}
```

● نکته

فیلدهایی که با `_` شروع می‌شوند (مثل `_hasHydrated`) یک قرارداد رایج برای State داخلی است. چون در `partialize` ذخیره نشده‌اند، بعد از هر رفرش دوباره `false` می‌شوند و چرخه تمیز است.

Redux DevTools با دیباگ – devtools

میان‌افزار `devtools` استور شما را به افزونه **Redux DevTools** (همان که برای Redux استفاده می‌شود) وصل می‌کند: هر یک اکشن در تایم‌لاین می‌شود، می‌توانید State هر لحظه را ببینید و حتی به عقب برگردید:

نام‌گذاری اکشن‌ها + devtools اتصال

JS

```
import { create } from 'zustand'
import { devtools } from 'zustand/middleware'

const useCartStore = create(
  devtools(
    (set) => ({
      items: [],
      addItem: (product) =>
        set(
          (s) => ({ items: [...s.items, product] }),
          undefined, // در replace جای
          'cart/addItem' // نام اکشن در DevTools
        ),
    }),
    { name: 'CartStore' } // نام استور در DevTools
  )
)
```

در محیط تولید می‌توانید شرطی وصل کنید تا هزینه اضافه ندهد:

development فقط در

JS

```
const wrap = import.meta.env.DEV ? devtools : (f) => f
const useStore = create(wrap((set) => ({ /* ... */ })))
```

immer – بنویس ساده، خودش immutable می‌کند

یادتان هست آپدیت تو در تو چه دردسری بود؟ میان‌افزار `immer` اجازه می‌دهد داخل `set` مثل کد معمولی «جهش» بنویسید و خودش نسخهٔ immutable می‌سازد:

با immer

JS

```
set((s) => {
  const item = s.cart.items.find((i) => i.id
  === id)
  if (item) item.qty += 1
})
```

immer بدون

JS

```
set((s) => ({
  cart: {
    ...s.cart,
    items: s.cart.items.map((i) =>
      i.id === id ? { ...i, qty: i.qty + 1 }
    : i
  ),
}))
```

● هشدار

با immer فقط داخل `set` می‌توانید جهش کنید؛ بیرون از آن State همچنان freeze شده و خواندنش مشکلی ندارد اما تغییرش خطا می‌دهد. ضمناً در immer `set` دار، چیزی return نکنید – یا جهش کنید یا مقدار برگردانید، نه هر دو.

middleware سفارشی: چطور ساخته می‌شود؟

هر middleware فقط تابعی است که `create` بعدی رامی گیرد و نسخهٔ مزین‌شده بوی گرداند. یک لاگر ساده بنویسیم تا جعبهٔ جادویی باز شود:

src/store/middleware/logger.js

JS

```
export const logger = (config) => (set, get, api) =>
  config(
    (args, ...rest) => {
      console.groupCollapsed('action', rest[2] ?? '(unnamed)')
      console.log('prev state:', get())
      set(args, ...rest)
      console.log('next state:', get())
      console.groupEnd()
    },
    get,
    api
  )

// استفاده: create(logger((set) => ({ ... })))
```

● یادداشت

میان‌افزارهای دیگری هم در اکوسیستم هست – `subscribeWithSelector` برای `subscribe` با سلکتور (فصل ۶)، `combine` برای ادغام State و اکشن‌های جدا، و بسته `zustand-middleware-yjs` برای همگام‌سازی چندکاربره. الگو همه یکسان است.

جمع‌بندی فصل

- middleware تابعی دور `create` است؛ ترتیب استاندارد: `persist` ← `devtools`.
- `persist` با `name` و بهتر است با `partialize`؛ برای تغییر ساختار، `migrate` + `version`.
- بازیابی غیرهمزمان است؛ با فلگ `hydration` از رندر «سبک خالی» جلوگیری کنید.
- `devtools` اکشن‌ها را در Redux DevTools می‌بیند؛ نام اکشن‌ها را در آرگومان سوم `set` بدهید.
- `immer` جهش امن داخل `set` می‌دهد؛ یا جهش یا `return`، نه هر دو.
- middleware سفارشی = تابعی که `config` را می‌گیرد و `set/get` را مزمین می‌کند.

● تمرین فصل ۵

استور تمرین فصل ۲ (`useTodoStore`) را ارتقا دهید: با `persist` ذخیره‌اش کنید (فقط آرایه `todos`). با `immer` اکشن‌ها را بازنویسی کنید و با `devtools` نام هر اکشن را ببینید. بعد ساختار `todo` را تغییر دهید (فیلد `priority` اضافه کنید) و `version: 2` و `migrate` بنویسید تا داده قدیمی تان سالم بماند. `DevTools` → تب `Application` → `Local Storage` را چک کنید.

الگوهای حرفه‌ای TypeScript – Zustand و اسلایس‌ها

از این فصل به بعد مثل یک تیم واقعی کد می‌زنیم: استورهای تایپ‌دار با TypeScript، الگوی اسلایس برای استورهای بزرگ، subscribe با سلکتور و ملاحظات SSR.

در این فصل یاد می‌گیرید

- فرم کُرید (curried) تایپ‌دار `create<State>()(...)`
- الگوی Slices برای شکستن استورهای بزرگ
- `subscribeWithSelector` و کاربردهای عملی
- ساختار پوشه‌بندی استور در پروژه‌های واقعی و نکات SSR

Zustand با TypeScript

در Zustand نسخه ۵، فرم استاندارد نوشتن استور تایپ‌دار «کُرید» است – یعنی دو بار پرانتز. دلیلش این است که TypeScript برای اینکه هم State و هم اکشن‌ها را درست حدس بزند، باید تایپ را «قبل از» دیدن تابع شما بداند:

src/store/useCounterStore.ts

TS

```
import { create } from 'zustand'

// داده + اکشن‌ها State: تایپ کل (۱)
type CounterState = {
  count: number
  increment: (by?: number) => void
  decrement: (by?: number) => void
  reset: () => void
}

// پرانتز اول؛ بعد تابع ← create<CounterState>() (۲)
export const useCounterStore = create<CounterState>()((set) => ({
  count: 0,
  increment: (by = 1) => set((s) => ({ count: s.count + by })),
  decrement: (by = 1) => set((s) => ({ count: s.count - by })),
  reset: () => set({ count: 0 }),
}))
```

از این لحظه اگر `increment(2)` را با آرگومان رشته صدا بزنید یا فیلدی را تایپ اشتباه بخوانید، کامپایلر خطا می‌دهد – همان چیزی که در تیم‌ها جان شما را نجات می‌دهد.

● هشدار – پرانتز جا نماند!

فرم اشتباه: `create<CounterState>((set) => ...)` (بدون پرانتز وسط). در این شکل استنتاج تایپ اکشن ها درست کار نمی کند و در مکن های استفاده خطای عجیب می گیرید. یادتان باشد: `create<State>()` سپس `((set) => ...)`.

تایپ های کمکی برای persist

وقتی میان افزار هم هست، زنجیره تایپ به ترکیب میان افزارها وابسته است؛ برای همین به جای نوشتن همه چیز دستی، از همان فرم کپی استفاده می کنیم و اگر لازم شد، تایپ State را با `Partialize` محدود می کنیم:

تایپ دار persist

TS

```
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

type CartState = {
  items: { id: number; qty: number }[]
  addItem: (id: number) => void
}

export const useCartStore = create<CartState>()(
  persist(
    (set) => ({
      items: [],
      addItem: (id) =>
        set((s) => ({ items: [...s.items, { id, qty: 1 }] })),
    }),
    {
      name: 'cart',
      // ذخیره شود؛ تایپ همین را تحمیل می کند فقط
      partialize: (s) => ({ items: s.items }),
    }
  )
)
```

الگوی Slices: استور بزرگ را بشکنید

اگر همه اپ را در یک استور بریزید (کاربر، سبد، فیلترها، نوتیفیکیشن...)، فایل صدها خط می شود و هر تغییر، rebase دردناک می سازد. الگوی Slices راه حل رسمی Zustand است: هر دامنه یک «اسلایس» مستقل، و ترکیب آن ها در یک استور واحد. با یک تایپ کمکی شوع می کنیم:

src/store/createStore.ts — تایپ کمکی

TS

```
import { StateCreator } from 'zustand'

// اسلایس ها به هم اشاره می کنند؛ یک نوع مشترک برای «کل استور» تعریف می کنیم
export type BoundStore = CartSlice & UISlice
```

```
export type SliceCreator<T> = StateCreator<
  BoundStore,    // دیدن همه چیز (برای get استور نهایی)
  [],            // میان افزارهای سفارشی
  [],            // میان افزارهای ماقبل
  T              // تایپ همین اسلایس
>
```

src/store/slices/cartSlice.ts

TS

```
import { SliceCreator } from '../createStore'

export type CartSlice = {
  items: { id: number; qty: number }[]
  addItem: (id: number) => void
  removeItem: (id: number) => void
  clear: () => void
}

export const createCartSlice: SliceCreator<CartSlice> = (set, get) => ({
  items: [],
  addItem: (id) =>
    set((s) => {
      const found = s.items.find((i) => i.id === id)
      if (found) {
        return { items: s.items.map((i) => (i.id === id ? { ...i, qty: i.qty + 1 } : i)) }
      }
      return { items: [...s.items, { id, qty: 1 }] }
    }),
  removeItem: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),
  clear: () => set({ items: [] }),
})
```

src/store/slices/uiSlice.ts

TS

```
import { SliceCreator } from '../createStore'

export type UISlice = {
  cartOpen: boolean
  toggleCart: (open?: boolean) => void
}

export const createUISlice: SliceCreator<UISlice> = (set) => ({
  cartOpen: false,
  toggleCart: (open) => set((s) => ({ cartOpen: open ?? !s.cartOpen })),
})
```

src/store/useBoundStore.ts — ترکیب نهایی

TS

```
import { create } from 'zustand'
import { createCartSlice } from './slices/cartSlice'
```

```
import { createUISlice } from './slices/uiSlice'
import { BoundStore } from './createStore'

export const useBoundStore = create<BoundStore>()((...a) => ({
  ...createCartSlice(...a),
  ...createUISlice(...a),
}))
```

حالا یک استور واحد دارید که هر اسلایس فایل خودش را دارد؛ و پاداش بزرگ: اسلایس‌ها به State همدیگر هم دسترسی دارند (از طریق `get`). مثلاً `addItem` می‌تواند بعد از افزودن، `toggleCart(true)` صدا بزند.

● نکته – یک استور بزرگ یا چند استور کوچک؟

هر دو الگو قابل قبول است. اگر بخش‌ها مستقل‌اند (تم و سبد هیچ ربطی به هم ندارند) چند استور جدا بسازید. اگر باید با هم تعامل کنند یا منطق مشترک دارند، اسلایس. آنچه ممنوع است، «استور خدای ۱۵۰۰ خطی» است.

subscribeWithSelector: گوش دادن هدفمند

`subscribe` پیش‌فرض به هر تغییر گوش می‌دهد. میان‌افزار `subscribeWithSelector` نسخهٔ سلکتوردار می‌دهد – مثل `useEffect` اما بدون رندر:

به یک فیلد خاص subscribe

```
import { subscribeWithSelector } from 'zustand/middleware'

const useBoundStore = create<BoundStore>()(
  subscribeWithSelector((...a) => ({ /* اسلایس‌ها */ }))
)

// مثلاً App):
useEffect(() => {
  const unsubs = useBoundStore.subscribe(
    (s) => s.items.length, // فقط این مقدار مهم است
    (len, prevLen) => { // فقط وقتی عوض شود صدا می‌شود
      if (len > prevLen) toast('به سبد اضافه شد')
    },
    { equalityFn: Object.is } // اختیاری
  )
  return unsubs
}, [])
```

کاربردهای واقعی: نمایش `toast` بعد از افزودن به سبد، ذخیرهٔ خودکار `draft` بعد از تغییر متن، گزارش `analytics` روی اکشن‌های خاص.

```
src/
├── store/
│   ├── useBoundStore.ts      # ترکیب اسلایس‌ها (اگر الگوی اسلایس)
│   └── slices/
│       ├── cartSlice.ts
│       └── uiSlice.ts
│   └── useAuthStore.ts      # استورهای مستقل دامنه‌ای
├── features/                # کامپوننت‌ها کنار دامنه خودشان
│   ├── cart/
│   └── products/
└── lib/                     # کد عمومی (api, utils)
```

- **SSR / Next.js:** `localStorage` روی سرور وجود ندارد. اگر `persist` دارید، آپشن `skipHydration: true` بدهید و بعداً در سمت کلاینت با `useCartStore.persist.rehydrate()` ادامه دهید. در SPA معمولی این موضوع ندارید.
- **Testability:** چون استور شیء معمولی است، در تست‌ها با `useXStore.setState({...})` مستقیم سناریو می‌سازید – فصل ۱۴ کاملش می‌کند.
- **قانون تنهایی:** استور نباید `fetch` بلد باشد مگر برای فرمان‌های دستورالعملی؛ خواندن داده سرور کار `Query` است (فصل بعدی).

جمع‌بندی فصل

- TS با فرم `create<State>()((set) => ...)` – پرانتز دوم را فراموش نکنید.
- الگوی اسلایس = هر دامنه یک فایل + تایپ مشترک + ترکیب در یک استور.
- `subscribeWithSelector` گوش دادن بدون رندر به یک فیلد خاص می‌دهد.
- در SSR با `persist` از `skipHydration` استفاده کنید؛ در SPA نیازی نیست.
- چند استور مستقل برای دامنه‌های بی‌ربط؛ اسلایس برای دامنه‌های متعامل.

تمرین فصل ۶

استور تم (`useThemeStore`) را به TypeScript مهاجرت کنید با تایپ دقیق `'light' | 'dark'` برای تم. بعد استور سبد و استور `UI` مودال را با الگوی اسلایس در یک استور ترکیب کنید به طوری که `addItem` بعد از هر افزودن، مودال سبد را باز کند – بدون هیچ رفتاری بین دو اسلایس.

پروژه عملی ۱ — سبد خرید کامل با Zustand

همهٔ آموخته‌های پنج فصل را در یک پروژه واقعی جمع می‌کنیم: فروشگاه کوچکی با گرید محصولات، سبد خرید کشویی، کد تخفیف، محاسبات مشتق‌شده و persist.

در این فصل یاد می‌گیرید

- طراحی استور برای یک دامنه واقعی قبل از نوشتن حتی یک خط UI
- جداسازی State داده از State UI
- محاسبات مشتق‌شده (جمع، تخفیف، نهایی) بدون تکرار داده
- اتصال persist و مدیریت hydration در یک پروژه کامل

طراحی قبل از کد

درخواست محصول: «کاربر محصولات را می‌بیند، به سبد اضافه می‌کند، تعداد را تغییر می‌دهد، کد تخفیف می‌زند و جمع نهایی را می‌بیند. سبد بعد از رفرش باقی بماند.» قبل از هر کدی، State را طراحی می‌کنیم:

داده	جایگاه	دلیل
<code>products</code> (کاتالوگ)	فایل ثابت <code>data/products.js</code>	دادهٔ نمایشی؛ بعداً (فصل ۱۳) می‌شود Server State
<code>items</code> ، <code>coupon</code>	<code>useCartStore</code>	Client Data State — مالکش مرورگر است
<code>cartOpen</code> ، <code>toast</code>	<code>useShopUIStore</code>	UI State — با دادهٔ سبد بی‌ربط
جمع‌ها و تخفیف	هیچ‌جا ذخیره نمی‌شود	مشتق است؛ با <code>useMemo</code> در کامپوننت حساب می‌شود

● نکته طراحی

در `items` فقط `id` و `qty` نگه می‌داریم، نه کل شیء محصول. اگر قیمت در کاتالوگ عوض شود، سبد خودکار با قیمت جدید حساب می‌شود و داده تکراری نمی‌ماند. (همان اصل Normalization در دیتابیس‌ها!)

۱) دادهٔ کاتالوگ

src/data/products.js

JS

```
export const PRODUCTS = [
  { id: 1, name: 'هدفون بی‌سیم Pro', price: 2_450_000, emoji: '🎧' },
  { id: 2, name: 'کیبورد مکانیکال ۶۸%', price: 1_890_000, emoji: '⌨️' },
  { id: 3, name: 'ماوس ارگونومیک M5', price: 980_000, emoji: '🖱️' },
  { id: 4, name: 'پایه لپ‌تاپ آلومینیومی', price: 720_000, emoji: '💻' },
]

export const COUPONS = {
```

```

YALDA10: { type: 'percent', value: 10, min: 2_000_000 }, // اگر بالای ۲ میلیون ۱۰٪
SHIP500: { type: 'fixed', value: 500_000, min: 1_000_000 }, // هزار تومان کمتر ۵۰۰
}

```

۲) استور سبد – قلب پروژه

src/store/useCartStore.js

JS

```

import { create } from 'zustand'
import { persist } from 'zustand/middleware'
import { COUPONS } from '../data/products'

const initialState = { items: [], couponCode: null, _hasHydrated: false }

export const useCartStore = create(
  persist(
    (set, get) => ({
      ...initialState,

      addItem: (id) =>
        set((s) => {
          const found = s.items.find((i) => i.id === id)
          if (found) {
            return { items: s.items.map((i) => (i.id === id ? { ...i, qty: i.qty + 1 } : i)) }
          }
          return { items: [...s.items, { id, qty: 1 }] }
        }),

      setQty: (id, qty) =>
        set((s) => ({
          items:
            qty <= 0
              ? s.items.filter((i) => i.id !== id)
              : s.items.map((i) => (i.id === id ? { ...i, qty } : i)),
        })),

      removeItem: (id) => set((s) => ({ items: s.items.filter((i) => i.id !== id) })),

      applyCoupon: (code) => {
        const coupon = COUPONS[code.trim().toUpperCase()]
        if (!coupon) return { ok: false, message: 'کد تخفیف نامعتبر است' }
        const subtotal = get().items.reduce((sum, i) => sum + i.qty * priceOf(i.id), 0)
        if (subtotal < coupon.min) {
          return { ok: false, message: 'حداقل مبلغ برای این کد برآورده نشده' }
        }
        set({ couponCode: code.trim().toUpperCase() })
        return { ok: true, message: 'کد تخفیف اعمال شد' }
      },

      removeCoupon: () => set({ couponCode: null }),
      clearCart: () => set({ items: [], couponCode: null }),
    })
  )
)

```

```

    setHasHydrated: (v) => set({ _hasHydrated: v }),
  }},
  {
    name: 'shop-cart',
    partialize: (s) => ({ items: s.items, couponCode: s.couponCode }),
    onRehydrateStorage: () => (state) => state?.setHasHydrated(true),
  }
)
)

// کمکی: قیمت محصول از کاتالوگ (منبع واحد حقیقت)
import { PRODUCTS } from '../data/products'
const priceOf = (id) => PRODUCTS.find((p) => p.id === id)?.price ?? 0

```

به `applyCoupon` دقت کنید: از `get()` برای خواندن جمع فعلی استفاده کرد و «نتیجۀ اعتبارسنجی» را به‌صورت مقدار برگرداند – کامپوننت می‌تواند بر اساس آن پیام نشان دهد؛ به‌جای اینکه منطق پیام‌ها داخل استور بنشیند.

۳) استور UI

src/store/useShopUIStore.js

JS

```

import { create } from 'zustand'

export const useShopUIStore = create((set) => ({
  cartOpen: false,
  openCart: () => set({ cartOpen: true }),
  closeCart: () => set({ cartOpen: false }),
}))

```

۴) کامپوننت‌ها

src/components/ProductGrid.jsx

JSX

```

import { PRODUCTS } from '../data/products'
import { useCartStore } from '../store/useCartStore'
import { useShopUIStore } from '../store/useShopUIStore'

export default function ProductGrid() {
  const addItem = useCartStore((s) => s.addItem)
  const openCart = useShopUIStore((s) => s.openCart)

  const handleAdd = (id) => {
    addItem(id)
    openCart()
  }

  return (
    <div className="grid">
      {PRODUCTS.map((p) => (

```

```

    <div key={p.id} className="card">
      <span className="emoji">{p.emoji}</span>
      <h3>{p.name}</h3>
      <p>{p.price.toLocaleString('fa-IR')} تومان</p>
      <button onClick={() => handleAdd(p.id)}>افزودن به سبد</button>
    </div>
  )}
</div>
)
}

```

src/components/CartDrawer.jsx

JSX

```

import { useMemo, useState } from 'react'
import { PRODUCTS } from '../data/products'
import { useCartStore } from '../store/useCartStore'
import { useShopUIStore } from '../store/useShopUIStore'

const byId = Object.fromEntries(PRODUCTS.map((p) => [p.id, p]))

export default function CartDrawer() {
  const cartOpen = useShopUIStore((s) => s.cartOpen)
  const closeCart = useShopUIStore((s) => s.closeCart)
  const items = useCartStore((s) => s.items)
  const couponCode = useCartStore((s) => s.couponCode)
  const setQty = useCartStore((s) => s.setQty)
  const removeItem = useCartStore((s) => s.removeItem)
  const clearCart = useCartStore((s) => s.clearCart)
  const applyCoupon = useCartStore((s) => s.applyCoupon)
  const removeCoupon = useCartStore((s) => s.removeCoupon)
  const hasHydrated = useCartStore((s) => s._hasHydrated)

  const [code, setCode] = useState('')
  const [msg, setMsg] = useState(null)

  // عوض شود couponCode یا items محاسبات مشتق شده - فقط وقتی
  const totals = useMemo(() => {
    const subtotal = items.reduce((sum, i) => sum + byId[i.id].price * i.qty, 0)
    let discount = 0
    if (couponCode === 'YALDA10') discount = Math.round(subtotal * 0.1)
    if (couponCode === 'SHIP500') discount = 500_000
    return { subtotal, discount, payable: subtotal - discount }
  }, [items, couponCode])

  if (!cartOpen) return null

  return (
    <aside className="drawer">
      <header>
        <h2>سبد خرید ({items.length.toLocaleString('fa-IR')})</h2>
        <button onClick={closeCart}>بستن</button>

```

```

</header>

{!hasHydrated && <p>در حال بازیابی سبد</p>}

{items.map((i) => (
  <div key={i.id} className="row">
    <span>{byId[i.id].emoji} {byId[i.id].name}</span>
    <span>
      <button onClick={() => setQty(i.id, i.qty - 1)}>-</button>
      {' '} {i.qty.toLocaleString('fa-IR')} {' '}
      <button onClick={() => setQty(i.id, i.qty + 1)}>+</button>
    </span>
    <button onClick={() => removeItem(i.id)}>حذف</button>
  </div>
)}}

{msg && <p className={msg.ok ? 'ok' : 'err'}>{msg.message}</p>}

{couponCode ? (
  <button onClick={removeCoupon}>حذف کد {couponCode}</button>
) : (
  <div className="coupon">
    <input value={code} onChange={(e) => setCode(e.target.value)} placeholder="کد تخفیف" />
    <button onClick={() => setMsg(applyCoupon(code))}>اعمال</button>
  </div>
)}

<footer>
  <div>جمع: {totals.subtotal.toLocaleString('fa-IR')}</div>
  <div>تخفیف: {totals.discount.toLocaleString('fa-IR')}</div>
  <div>قابل پرداخت: <b>{totals.payable.toLocaleString('fa-IR')}</b></div>
  <button disabled={items.length === 0} onClick={clearCart}>پرداخت و خالی کردن سبد</button>
</footer>
</aside>
)
}

```

نقاط آموزشی پروژه

- دو استور، دو دنیا: `useCartStore` داده است و `useShopUIStore` فقط نمایش. کامپوننت گرید محصول از دومی فقط «باز کردن» می گیرد؛ از محتوای سبد خبر ندارد.
- سلکتور تکی همه جا: هر خط `useXStore((s) => ...)` یک فیلد است — دقیقاً همان که فصل ۴ گفت.
- مرجع پایدار برای `useMemo`: چون `items` فقط هنگام mutation جدید می شود، `totals` کم محاسبه می شود.
- **hydration** فلگ: پیام «در حال بازیابی» تا تمام شدن `rehydrate` رندر می شود و بعد لیست واقعی.

● هشدار – مرحله بعدی همان تله کلاسیک است

اگر همین پروژه بخواهد «موجودی واقعی» را از سرور بگیرد، اولین وسوسه این است که موجودی را در `useCartStore` ذخیره کنیم! از فصل ۸ می بینیم چرا داده سرور باید در کشی TanStack Query برود و Zustand فقط «سبد خود کاربر» را نگه دارد.

جمع بندی فصل

- اول State را روی کاغذ طراحی کنید؛ بعد UI را بنویسید.
- در سبد فقط id و qty نگه دارید؛ جزئیات را از منبع اصلی بخوانید.
- محاسبات مشتق شده ذخیره نمی شوند؛ با useMemo در کامپوننت حساب می شوند.
- اعتبارسنجی (کد تخفیف) داخل اکشن با `get()` و برگرداندن نتیجه به کامپوننت.
- persist با partialize + فلگ hydration برای تجربه بدون فلش.

● تمرین فصل ۷ – سه ارتقا

(۱) قابلیت «ذخیره برای بعد» اضافه کنید: لیست علاقه مدی ها (wishlist) در استور جدا با persist مشترک؟ نه – استور مستقل با نام کلید خودش. (۲) دکمه «خالی کردن» با تأیید دو مرحله ای (confirming State در UI استور). (۳) کد تخفیف جدید بنویسید که اگر تعداد کل آیتم ها از ۵ بیشتر بود ۱۵٪ بدهد – سرنخ: در `applyCoupon` به `items.length` نیاز دارید.

Server State و دنیای TanStack Query

چرا دادهٔ سرور با State معمولی فرق دارد؟ در این فصل مرز Client State و Server State را قطعی می‌کنیم، معماری کش TanStack Query را می‌شناسیم و پروژهٔ آماده می‌کنیم.

در این فصل یاد می‌گیرید

- پنج ویژگی Server State که آن را از Client State جدا می‌کند
- نصب و راه‌اندازی `QueryClientProvider` و تنظیمات پیش‌فرض تیمی
- نقشهٔ معماری: `queryFn`، `queryKey`، کش، `observer` و `garbage collection`
- واژگان کلیدی که تا آخر دوره بارها می‌بینید

چرا نمی‌توان دادهٔ سرور را مثل State معمولی مدیریت کرد؟

سبد خرید مال شماست: فقط شما آن را می‌بینید و فقط شما تغییرش می‌دهید. اما «لیست محصولات» را نه — هر لحظه ممکن است سرور آن را عوض کند، همکار دیگری تغییرش داده باشد و شما اصلاً خبر ندارید. این تفاوتِ ماهوی است، نه فنی. پنج ویژگی Server State:

۱. **دور و مشترک:** جای ذخیرهٔ اصلی جای دیگری است و ممکن است بدون اطلاع شما تغییر کند.
۲. **به‌روزرسانی‌پذیر از بیرون:** دیگر کاربران، پنل ادمین، حتی زمان (موجودی و قیمت‌ها).
۳. **نیازمند چرخهٔ `async`:** هر دسترسی یعنی شبکه با تأخیر، خطا و تلاش مجدد.
۴. **نیازمند حالت‌های فرآیندی:** در حال بارگذاری، خطا، موفقیت، State — `stale/fresh` معمولی این مفاهیم را ندارد.
۵. **کش‌شدنی اما قابل کهنگی:** نسخهٔ فعلی فقط یک snapshot است؛ ممکن است کهنه شود.

ویژگی	Client State (Zustand)	Server State (TanStack Query)
مالک داده	مرورگر کاربر	سرور / دیتابیس
ماهیت ذخیره	منبع حقیقت (Source of Truth)	کش (Cache) از دادهٔ دیگران
تغییر	با اکشن‌های خودمان	با <code>mutation + invalidation</code>
قابلیت کهنگی	معنا ندارد	مفهوم محوری (<code>stale/fresh</code>)
ری‌فرش خودکار	معنا ندارد	با فوکوس، ری‌کانکت و <code>interval</code>

● ممنوع — دو منبع حقیقت

هرگز پاسخ سرور را در Zustand کپی نکنید (`useUserStore.setState({ profile: data })`). از آن لحظه دو نسخه از یک داده دارید و دیر یا زود ناهماهنگ می‌شوند و بگ‌های «گاهی درست است، گاهی نه» می‌سازید. دادهٔ سرور فقط در کش Query زندگی می‌کند.

terminal

BASH

```
npm install @tanstack/react-query
npm install -D @tanstack/react-query-devtools
```

src/main.jsx

JSX

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { ReactQueryDevtools } from '@tanstack/react-query-devtools'
import App from './App'

// برای کل اپ - بیرون از کامپوننت بسازید QueryClient یک ()
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60_000, // داده تا ۱ دقیقه «تازه» محسوب شود
      retry: 2, // دو بار تلاش مجدد قبل از اعلام خطا
      refetchOnWindowFocus: true, // برگشت به تب = بررسی تازگی
    },
  },
})

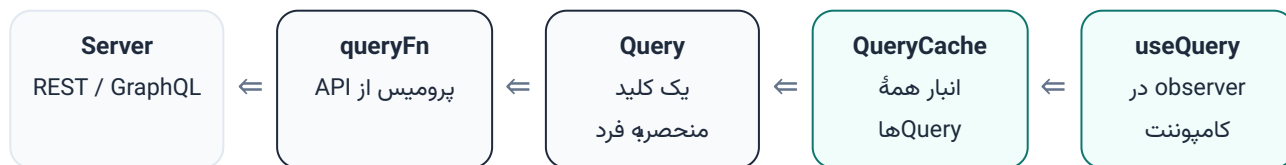
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
      <ReactQueryDevtools initialIsOpen={false} />
    </QueryClientProvider>
  </React.StrictMode>
)
```

QueryClient مغز متفکر است: کش، پیکربندی و ابزارهای دستوری (invalidate، prefetch و...) همه در آن‌اند و به‌صورت یکتا در کل اپ وجود دارد. **QueryClientProvider** همان Provider است که در Zustand نداشتیم – اینجا برای اشتراک یک client واحد لازم است.

● نکته – تنظیمات پیش‌فرض، جایی که تصمیم‌های تیمی گرفته می‌شود

به جای اینکه هر **useQuery** جداگانه staleTime بدهد، در **defaultOptions** یک بار برای کل اپ تعیین کنید و فقط در موارد استثنا، سطح Query بازنویسی کنید. این همان کاری است که در code review از شما خواسته می‌شود.

معماری کش: پنج بازیگر اصلی



هر `useQuery` یک **observer** به یک `Query` در کش اضافه می کند. `Query` با **queryKey** هویت می گیرد (همان کلید دیکشنری). داده ای که `queryFn` بوی گرداند در همان `Query` ذخیره می شود. وقتی آخرین `observer` حذف شود (کامپوننت `unmount`)، داده تا پایان **gcTime** (پیش فرض ۵ دقیقه) برای استفاده های بعدی زنده می ماند و بعد از حافظه پاک می شود. دو زمان کلیدی:

پارامتر	پیش فرض	معنا
<code>staleTime</code>	0	چه مدت داده «تازه» است؛ در این مدت هیچ رفرشی به صورت خودکار رخ نمی دهد (پیش فرض ۰ یعنی بلافاصله <code>stale</code>).
<code>gcTime</code>	۵ دقیقه	چند وقت پس از بی <code>observer</code> شدن، داده از حافظه پاک شود. (در نسخه ۴ به اسم <code>cacheTime</code> بود).

● هشدار – `stale` یعنی «ممکن است کهنه باشد»، نه «غیرقابل استفاده»

داده `stale` فوراً از کش رندر می شود؛ فقط در پس زمینه هم `fetch` تازه اجرا می شود و وقتی رسید، `UI` بی دردرسر آپدیت می شود. الگوی «نمایش فوری کش + روزرسانی در پس زمینه» همان چیزی است که `پ` ها را «سریع» حس می کند.

واژگانی که تا آخر دوره همراه ماند

- **Query**: خواندن اعلامی از داده سرور (GET-مانند) با کلید یکتا.
- **Mutation**: تغییر داده سرور (POST/PUT/DELETE) – اعلامی نیست؛ خودتان اجلیش می کنید.
- **Invalidation**: بعد از `mutation`، علامت گذاری `Query` های مرتبط به `stale` تا دوباره گرفته شوند – جایگزین دستی `sync` کردن.
- **Refetch**: گرفتن دوباره داده؛ خودکار (فوکوس/ری کانکت/interval) یا دستی.
- **Optimistic Update**: قبل از جواب سرور، `UI` را آپدیت کن؛ اگر شکست، برگرد (فصل ۱۰).

از نسخه ۴ به ۵ – اگر جایی که قدیمی دیدید

در نسخه ۴	در نسخه ۵
<code>useQuery({ ... }, queryFn)</code> یا آرگومان‌های جدا	فقط شیء: <code>useQuery({ queryKey, queryFn })</code>
<code>isLoading</code> = هنوز داده‌ای نیست	<code>isLoading</code> ؛ <code>isPending</code> حالا = در حال <code>fetch</code> بدون داده اولیه
<code>cacheTime</code>	<code>gcTime</code>
<code>keepPreviousData: true</code>	<code>placeholderData: keepPreviousData</code> (فصل ۱۲)
<code>useQuery(..., { enabled })</code>	همه در یک شیء: <code>useQuery({ ..., enabled })</code>
<code>isFetchingNextPage</code> بدون تایپ	همان است ولی همه هوک‌ها تایپ عمومی بهتر دارند و TS-only شد

پروژه بخش دوم

در ادامه دوره یک API عمومی به اسم **JSONPlaceholder** را مبنای تمرین می‌کنیم (<https://jsonplaceholder.typicode.com>) – سرور آزمایشی با endpoint های `/users` ، `/posts` و `/comments` که همیشه در دسترس است و write‌های ساختگی انجام می‌دهد. برای پروژه فصل ۱۳ همین را به‌عنوان Backend استفاده می‌کنیم؛ در پروژه‌های واقعی فقط آدرس عوض می‌شود.

یادداشت

TanStack Query فریم‌وک محور نیست: هسته آن ([@tanstack/query-core](https://github.com/tannerkristof/tanstack-query-core)) با Vue و Solid و Svelte هم کار می‌کند. هر چیزی که اینجا یاد می‌گیرید، دانش قالی حمل است.

جمع‌بندی فصل

- Server State داده‌ای است که مالکش شما نیست؛ مدیریتش یعنی مدیریت یک کش.
- معماری: Query → QueryCache → observer (useQuery) (با queryKey) → queryFn → سرور.
- `staleTime` = مدت تازگی؛ `gcTime` = مدت زنده ماندن بعد از `unmount`.
- تنظیمات پیش‌فرض را یک‌جا در `defaultOptions` بدهید.
- داده سرور هرگز به Zustand کپی نمی‌شود.

تمرین فصل ۸

پروژه Vite بخش اول را با [@tanstack/react-query](https://github.com/tannerkristof/tanstack-react-query) راه‌اندازی کنید: QueryClient با ۳۰ `staleTime` ثانیه و DevTools. در App یک کامپوننت آزمایشی بسازید که با `fetch` آدرس <https://jsonplaceholder.typicode.com/users> را صدا بزند (فعلاً همان `useEffect` قدیمی – فصل بعد اصلش می‌کنیم) و خروجی را در کنسول چاپ کند. فقط مطمئن شوید اپ بدون خطا بالا می‌آید.

useQuery — خواندن داده از سرور

هوکی که جای همهٔ `loading/error/useEffect` دستی می‌نشیند: تعریف `query`، شناخت وضعیت‌ها، کنترل تازگی با `staleTime` و تنظیمات عملی `retry` و `cancellation`.

در این فصل یاد می‌گیرید

- نوشتن اولین `useQuery` با `queryKey` و `queryFn`
- تفکیک `status` و `fetchStatus` و رندر بر اساس آن‌ها
- رفتار کش در عمل: `dedup`، `refetch` و `staleTime`
- سلکت‌کردن و تبدیل داده با `select`، کوئری‌های وابسته با `enabled`

اولین useQuery

قرارداد پروژه: یک لایهٔ API جدا نگه می‌داریم تا `queryFn`‌ها فقط «فراخوانی» باشند:

src/lib/api.js

JS

```
const BASE = 'https://jsonplaceholder.typicode.com'

export async function getJSON(path) {
  const res = await fetch(BASE + path)
  if (!res.ok) throw new Error(`API error ${res.status}`)
  return res.json()
}

export const api = {
  posts: (params = '') => getJSON(`/posts${params}`),
  post: (id) => getJSON(`/posts/${id}`),
  users: () => getJSON('/users'),
}
```

src/components/PostsList.jsx

JSX

```
import { useQuery } from '@tanstack/react-query'
import { api } from '../lib/api'

export default function PostsList() {
  const { data, isPending, isError, error } = useQuery({
    queryKey: ['posts'],
    queryFn: () => api.posts('?_limit=10'),
  })

  if (isPending) return <p>در حال بارگذاری پست‌ها</p>
  if (isError) return <p>خطا: {error.message}</p>
```

```

return (
  <ul>
    {data.map((post) => (
      <li key={post.id}>
        <b>{post.id}. {post.title}</b>
      </li>
    ))}
  </ul>
)
}

```

دو ورودی، همه چیز:

- **queryKey**: هویت این داده در کش. آرایه است و از چپ به راست «از عمومی به خاص» خوانده می‌شود: `['posts']` لیست پست‌ها، `['posts', 5]` پست شماره ۵، `['posts', { page: 2, q: 'react' }]` لیست با پارامترها. هر تغییر در محتوای آرایه = یک داده متفاوت.
- **queryFn**: تابعی که Promise داده را بوی گرداند. اگر reject شود، Query به status خطا می‌رود و بعد از retry خطا در **error** است.

● هشدار — queryFn باید واقعاً throw کند

اگر در queryFn خطای HTTP را با `if (!res.ok) return null` «بلعید»، TanStack Query فکر می‌کند همه چیز موفق بوده و UI شما رندر خالی می‌شود. همیشه خطا را پرتاب کنید (همان کاری که `getJSON` بالا می‌کند).

وضعیت‌ها: status در برابر fetchStatus

مهم‌ترین نکته مفهومی فصل. هر Query دو چراغ دارد:

چراغ	مقادیر	سؤالی که جواب می‌دهد
status / هوک‌های <code>isLoading</code> · <code>isPending</code> · <code>isError</code> · <code>isSuccess</code>	<code>pending / error / success</code>	«آیا اصلاً داده‌ای داریم؟» — درباره خود داده
fetchStatus / <code>isFetching</code>	<code>fetching / idle / paused</code>	«همین الان درخواستی در جریان است؟» — درباره فرآیند شبکه

ترکیب‌های معنادار:

- `isPending && isFetching`: بارگذاری اولیه؛ صفحه loading نشان بدهید. (این ترکیب همان `isLoading` است.)
- `isSuccess && isFetching`: داده قدیمی روی صفحه + رفرش در پس‌زمینه؛ یک ایندیکاتور کوچک نشان دهید، صفحه را خالی نکنید!
- `isSuccess && !isFetching`: داده تازه و آرام.

- `isError`: خطا – اما داده قبلی ممکن است هنوز در `data` باشد.

● ممنوع – خالی کردن صفحه هنگام رفرش

یعنی هر برگشت کاربر به تب، کل صفحه به اسپینر تبدیل شود. برای رفرش پس زمینه فقط `isFetching` را به یک نشانگر کوچک وصل کنید؛ `isPending` مال صفحه loading اولیه است.

رفتار کش در عمل: `refetch` و `dedup`

سه کامپوننت مختلف، همان کلید:

یک درخواست، observer، سه

JSX

```
// در سه کامپوننت مختلف:
const { data } = useQuery({ queryKey: ['users'], queryFn: api.users })
// می‌گیرند data فقط «یک» درخواست شبکه زده می‌شود و هر سه همان
// Request Deduplication به این می‌گویند.
```

وقتی داده بوی گردد، کش آن را با کلید `['users']` ذخیره می‌کند. بعداً اگر کامپوننت دیگری mount شود، فوراً داده کش را می‌گیرد (`isSuccess` از اول) و Query طبق قوانین `refetch` تصمیم می‌گیرد دوباره بگیرد یا نه. رفرش خودکار سه‌گانه:

- `refetchOnWindowFocus` (پیش‌فرض روشن): برگشت به تب اپ → اگر داده stale است، رفرش پس‌زمینه.
- `refetchOnMount` (پیش‌فرض روشن): mount دوباره → اگر stale است، رفرش.
- `refetchOnReconnect` (پیش‌فرض روشن): برگشتن اینترنت → رفرش.

همه این‌ها با `staleTime` کنترل می‌شوند: تا داده «تازه» است، هیچ‌کدام اضافه‌کاری نمی‌کنند. تنظیم پیشنهادی تیمی: داده‌های کم‌تغییر (کشورها، لیست دسته‌ها) `staleTime` بالا (ساعت‌ها)، داده‌های زنده (قیمت، اعلان) `staleTime` کوتاه یا ۰.

Query تنظیمات سطح یک

JS

```
const { data: notifications } = useQuery({
  queryKey: ['notifications'],
  queryFn: () => api.get('/notifications'),
  staleTime: 10_000, // Query بازنویسی پیش‌فرض فقط برای همین
  refetchInterval: 30_000, // هر ۳۰ ثانیه polling
  refetchIntervalInBackground: false, // poll در تب پنهان
})
```

retry – تلاش مجدد هوشمند

پیش‌فرض: ۳ بار با تأخیر نمایی (۱ ثانیه، ۲، ۴). کنترل آن:

سفارشی retry

JS

```
useQuery({
  queryKey: ['posts'],
```

```

queryFn: () => api.posts(),
retry: 2,
// یا تابعی: خطای ۴۰۴ را نگیر (منبع واقعاً نیست)، بقیه را بگیر
retry: (failureCount, error) => {
  if (error.message.includes('404')) return false
  return failureCount < 3
},
retryDelay: (attempt) => Math.min(1000 * 2 ** attempt, 30_000),
})

```

لغو درخواست: signal

TanStack Query به هر queryFn یک `signal` (همان AbortController مرورگر) می‌دهد؛ وقتی Query بی‌observer شود، درخواست لغو می‌شود و شبکه اشغال نمی‌ماند. برای استفاده، signal را به fetch بدهید:

src/lib/api.js — نسخه کامل

JS

```

export const api = {
  posts: (params = '', signal) => getJSON(`/posts${params}`, signal),
  post: (id, signal) => getJSON(`/posts/${id}`, signal),
}

async function getJSON(path, signal) {
  const res = await fetch(BASE + path, { signal })
  if (!res.ok) throw new Error(`API error ${res.status}`)
  return res.json()
}

// در useQuery:
queryFn: ({ signal }) => api.posts('?_limit=10', signal)

```

● نکته

هر بار که کلید عوض شود (مثلاً کاربر سریع بین صفحات می‌زند)، درخواست قبلی به طور خودکار بی‌ارزش می‌شود؛ با signal، لغو هم واقعی می‌شود. رقابت پاسخ‌ها (race condition) در TanStack Query اساساً وجود ندارد: هر پاسخ فقط به کلید خودش ثبت می‌شود.

select: تبدیل داده در سطح کش

اگر فقط بخشی از داده لازم دارید (مثلاً نام کاربران از آرایه کامل)، `select` درست‌کار است — فقط هنگام تغییر «خروجی سلکت‌شده» زدر می‌گیرد:

خودکار memo + select

JSX

```

function UserNames() {
  const names = useQuery({
    queryKey: ['users'],
    queryFn: api.users,

```

```

    select: (users) => users.map((u) => u.name), // خروجی تبدیل شده
  })
  // select می شود؛ memo در نسخه ۵ به صورت داخلی
  // پایدارش کنید useMemo را بیرون تعریف کردید، با select اگر تابع
  return <ul>{names.data?.map((n) => <li key={n}>{n}</li>)}</ul>
}

```

کوئری های وابسته: enabled

کلاسیک ترین سناریو: «اول کاربر را بگیر، بعد پست های همان کاربر». تا id کاربر نیامده، کوئری دوم نباید اجرا شود:

```

src/components/UserPosts.jsx
JSX

import { useQuery } from '@tanstack/react-query'
import { api } from '../lib/api'

export default function UserPosts({ email }) {
  // کاربر از روی ایمیل (۱)
  const { data: user } = useQuery({
    queryKey: ['user', email],
    queryFn: ({ signal }) => api.findUserByEmail(email, signal),
  })

  const userId = user?.id

  // آماده است userId پست ها - فقط وقتی (۲)
  const postsQuery = useQuery({
    queryKey: ['posts', { userId }],
    queryFn: ({ signal }) => api.posts(`?userId=${userId}`, signal),
    enabled: !!userId, // fetch ممنوع تا userId نباشد
  })

  if (!user) return <p>در حال پیدا کردن کاربر</p>
  return <PostList data={postsQuery.data ?? []} />
}

```

● هشدار – کلید، فقط داده پایدار

در queryKey هر چیزی که serialize شدن است می گذارید (عدد، رشته، شیء ساده، آرایه). تابع یا کلاس نگذارید. کلیدهای به هم ریخته یعنی کش به هم ریخته – فصل ۱۱ کل ادبیات کلیدها را ساختار می دهد.

دو ابزار آخر این فصل: initialData و placeholderData

- **initialData**: داده اولیه قطعی – Query از همان لحظه اول «success» است (مثلاً داده SSR یا پیش فرض ثابت). فقط وقتی واقعاً از صحتش مطمئنید.
- **placeholderData**: داده «نمایشی» تا fetch واقعی بیاید؛ status می ماند pending و **isPlaceholderData: true**. کاربرد ستاره ای اش در صفحه بندی است (فصل ۱۲):

یعنی «تا دادهٔ صفحهٔ بعد نیامده، صفحهٔ قبل را نشان بده».

`placeholderData: keepPreviousData`

جمع‌بندی فصل

- `useQuery({ queryKey, queryFn })` ؛ کلید = هویت در کش، `queryFn = Promise` که `throw` می‌کند.
- `status` دربارهٔ داده است (`pending/error/success`)، `fetchStatus` دربارهٔ شبکه (`fetching/idle`).
- `dedup` خودکار، `refetch` خودکار با فوکوس/ری‌کانکت و کنترل با `staleTime`.
- `signal` را به `fetch` بدهید تا لغو واقعی داشته باشید؛ `race condition` حذف می‌شود.
- `select` برای تبدیل، `enabled` برای وابستگی، `placeholderData` و `initialData` برای تجربهٔ نرم.

● تمرین فصل ۹

کامپوننت `PostsList` را ارتقا دهید: (۱) سلکتور کاربر، آرایهٔ پست‌ها را بر اساس ورودی جستجو فیلتر کند و با `select` خروجی فیلترشده بگیرد. (۲) کوئری دوم بسازید که با کلید `['post', id]` جزئیات یک پست را بگیرد و با کلیک روی هر پست نمایش دهد (با ۳). (۳) `enabled` در `DevTools` ببینید وقتی دو بار روی همان پست کلیک می‌کنید، درخواست جدیدی زده نمی‌شود — چرا؟ (`staleTime` پیش فرض ۶۰ ثانیهٔ App را یادتان هست؟)

useMutation – تغییر داده و بهروزرسانی کش

ساخت، حذف و ویرایش روی سرور؛ چرخه mutation، اینولیدیشن به‌عنوان راهبرد اصلی sync، و بهروزرسانی خوش‌بینانه با rollback – الگویی که در مصاحبه‌ها پرسیده می‌شود.

در این فصل یاد می‌گیرید

- تعریف `useMutation` با `mutationFn` و فراخوانی با `variables`
- چرخه `onMutate` → `onSuccess` / `onError` → `onSettled`
- الگوی بهروزرسانی خوش‌بینانه کامل با `snapshot` و `rollback`
- انتخاب بین `setQueryData` و `invalidation`

آناتومی یک mutation

بر خلاف `useQuery` که هنگام mount خودش اجرا می‌شود، `mutation` دستی اجرا می‌شود – وقتی کاربر دکمه را زد:

src/lib/api.js – write اضافه شدن

JS

```
async function sendJSON(path, method, body) {
  const res = await fetch(BASE + path, {
    method,
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
  })
  if (!res.ok) throw new Error(`API error ${res.status}`)
  return res.json()
}

export const api = {
  // ... فصل ۹ پایین همان ...
  addPost: (post) => sendJSON('/posts', 'POST', post),
  updatePost: (id, patch) => sendJSON(`/posts/${id}`, 'PATCH', patch),
  deletePost: (id) => sendJSON(`/posts/${id}`, 'DELETE').then(() => true),
}
```

src/components/NewPostForm.jsx

JSX

```
import { useState } from 'react'
import { useMutation, useQueryClient } from '@tanstack/react-query'
import { api } from '../lib/api'

export default function NewPostForm() {
  const [title, setTitle] = useState('')
  const queryClient = useQueryClient()
```

```
const addPost = useMutation({
  mutationFn: (variables) => api.addPost(variables),
  onSuccess: (data, variables) => {
    // کن تا از سرور تازه شود stale لیست پست‌ها را (۱)
    queryClient.invalidateQueries({ queryKey: ['posts'] })
    setTitle('')
  },
  onError: (error, variables) => {
    alert('ذخیره نشد: ' + error.message)
  },
})

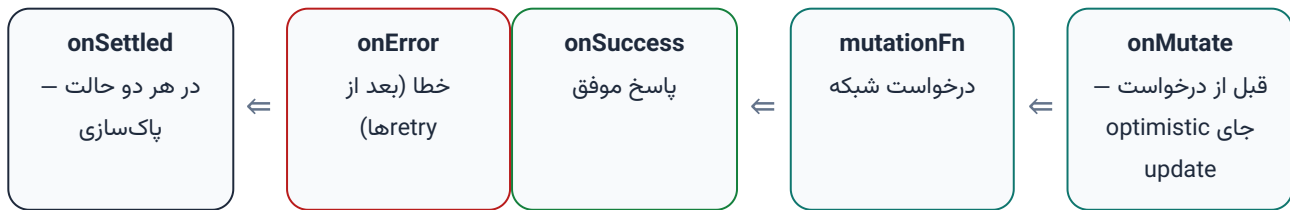
return (
  <form
    onSubmit={e => {
      e.preventDefault()
      addPost.mutate({ title, body: '...', userId: 1 })
    }}
  >
    <input value={title} onChange={e => setTitle(e.target.value)} />
    <button disabled={addPost.isPending}>
      {addPost.isPending ? 'انتظار' : 'ذخیره'}
    </button>
    {addPost.isError && <p className="err">{addPost.error.message}</p>}
  </form>
)
}
```

نقش	قطعه
تنها آرگومان اجباری؛ Promise که سرور را تغییر می‌دهد.	<code>mutationFn</code>
اجرای <code>mutation</code> : هر داده‌ای (فرم، <code>id</code> ، فایل) به‌عنوان <code>variables</code> به <code>mutationFn</code> و کالک‌ها می‌رسد.	<code>mutate(variables)</code>
وضعیت‌ها مثل <code>isPending</code> – <code>useQuery</code> یعنی درخواست در جریان است.	<code>isPending / isError / error / data</code>
دسترسی به <code>client</code> برای <code>invalidate</code> و <code>setQueryData</code> بعد از تغییر.	<code>useQueryClient()</code>

● هشدار – `mutate` یا `mutateAsync`؟

`mutate()` هیچ Promise‌ای بر نمی‌گرداند و خطاها را به `onError` می‌سپارد – انتخاب پیش فرض. اگر منطق بعد از اتمام را «آوانس محور» می‌خواهید (مثلاً `navigate` بعد از موفقیت)، از `const res = await mutateAsync(vars)` داخل `try/catch` استفاده کنید. هرگز `mutateAsync` را بدون `try/catch` صدا نزنید؛ `unhandled rejection` می‌گیرید.

چرخه کامل کالیک‌ها



ترتیب اجرا: کالیک‌های خود **mutation** اول اجرا می‌شوند و بعد کالیک‌های component-level همان فراخوانی (اگر در `mutate(vars, { onSuccess })` داده باشید). در تمرین‌های تیمی معمولاً منطق اصلی را در هوک می‌گذاریم و موارد UI-محلی (toast، بستن مودال) را در متد `mutate`.

invalidation: بعد از تغییر، داده را «کهنه» کن

دو راه برای به‌روز نگه‌داشتن کش بعد از mutation:

۱. **Invalidation (پیش‌فرض حرفه‌ای):** بگو «داده X دیگر قابل اعتماد نیست». Query‌های متناظر stale می‌شوند و اگر observer فعال داشته باشند بلافاصله `refetch` می‌گیرند.
۲. **setQueryData دستی:** پاسخ سرور را مستقیم در کش بنویس — سریع اما مسئولیت شکل‌دادن به داده با شماست.

دو الگو، پهلوی به پهلوی

JS

```
// ساده، همیشه درست - invalidation: الگوی ۱
onSuccess: () => {
  queryClient.invalidateQueries({ queryKey: ['posts'] })
}

// الگوی ۲: نوشتن مستقیم پاسخ سرور در کش
onSuccess: (saved) => {
  queryClient.setQueryData(['post', saved.id], saved)
  queryClient.invalidateQueries({ queryKey: ['posts'] }) // refetch لیست همچنان
}
```

انتخاب: برای «لیست‌ها» `invalidation` (سرور منبع حقیقت است)، برای «جزئیات همان آیتم» می‌توانید `setQueryData` (پاسخ را دارید، چرا دوباره بگیریم؟). ترکیب هر دو رایج و سالم است.

به‌روزرسانی خوش‌بینانه (Optimistic Update)

ایده: تغییر را فوراً در کش بنویس (خوش‌بین باش) تا UI آتی شود؛ اگر سرور جواب خطا داد، به حالت قبل برگرد. الگوی رسمی نسخه ۵:

حذف خوش‌بینانه — src/components/PostRow.jsx

JSX

```
import { useMutation, useQueryClient } from '@tanstack/react-query'
import { api } from '../lib/api'
```

```

export default function DeletePostButton({ id }) {
  const queryClient = useQueryClient()

  const removePost = useMutation({
    mutationFn: (id) => api.deletePost(id),

    // قبل از درخواست: نسخه فعلی را ذخیره کن و کنش را آپدیت کن (۱)
    onMutate: async (id) => {
      await queryClient.cancelQueries({ queryKey: ['posts'] }) // refetch کن متوقف کن جریان را متوقف کن
      const previous = queryClient.getQueryData(['posts']) // snapshot
      queryClient.setQueryData(['posts'], (old) =>
        old?.filter((p) => p.id !== id) ?? old
      )
      return { previous } // کالک‌های بعدی می‌رسد به context
    },

    // snapshot خطا: برگرد به (۲)
    onError: (err, id, context) => {
      queryClient.setQueryData(['posts'], context.previous)
      alert('حذف ناموفق بود؛ بازگردانی شد')
    },

    // کن sync در هر حالت: همیشه با سرور (۳)
    onSettled: () => {
      queryClient.invalidateQueries({ queryKey: ['posts'] })
    },
  })

  return (
    <button disabled={removePost.isPending} onClick={() => removePost.mutate(id)}>
      {removePost.isPending ? '...' : 'حذف'}
    </button>
  )
}

```

سه گام طلایی که حفظ کنید:

۱. `onMutate: cancel + snapshot + setQueryData` + بازگرداندن `{ previous }`.

۲. `onError`: بازگردانی از `context.previous`.

۳. `onSettled: invalidation` برای همگام‌شدن با حقیقت سرور.

● هشدار — `cancelQueries` را حذف نکنید

اگر قبل از `setQueryData`، یک `refetch` در جریان باشد، وقتی تمام شود داده قدیمی را روی تغییر خوش بینانه تان می نویسد و حذف آیتیم «برمی گردد». اول `cancel`، بعد `snapshot`، بعد `set`.

mutation های هماهنگ با هم: useMutationState

برای نمایش وضعیت‌های جهانی (مثلاً «N badge تغییر در حال ذخیره») لازم نیست وضعیت را جابه‌جا کنید؛ Query خودش همهٔ mutation ها را نگه می‌دارد:

های در جریان mutation شمارش

JSX

```
import { useMutationState } from '@tanstack/react-query'

function SavingBadge() {
  const saving = useMutationState({
    filters: { status: 'pending' },
    select: (m) => m,
  })
  if (saving.length === 0) return null
  return <span className="badge">{saving.length} تغییر ذخیره می‌شود </span>
}
```

جمع‌بندی فصل

- mutation دستی اجرا می‌شود: `useMutation({ mutationFn })` + `.mutate(variables)`
- چرخه: `.onMutate` → `mutationFn` → `onSuccess/onError` → `onSettled`
- بعد از هر `mutation: invalidation` (پیش‌فرض) یا `setQueryData` (پاسخ قطعی).
- الگوی خوش‌بینانه: `rollback` + `getQueryData/setQueryData` + `cancelQueries` در `onError` + `invalidate` در `onSettled`
- `useMutationState` وضعیت‌های سراسری را بدون state اضافه می‌دهد.

تمرین فصل ۱۰

۱) فرم ویرایش پست بسازید (PATCH با `api.updatePost`) که بعد از موفقیت، هم جزئیات `['post', id]` را با `setQueryData` آپدیت کند و هم لیست را `invalidate` کند. ۲) افزودن پست را خوش‌بینانه کنید: اول با `id` موقت (مثلاً `Date.now()`) در لیست بگذارید و بعد از موفقیت `invalidate` کنید. ۳) با DevTools ببینید بعد از هر `mutation`، `Query` های `stale` چطور دوباره `fetch` می‌شوند.

Query Keys و مدیریت کش

کلید، شناسنامه داده در کش است. در این فصل الگوی رسمی Query Key Factory، اینولیدیشن هدفمند با فیلترها، prefetch و ابزارهای دستوری QueryClient را یاد می‌گیرید.

در این فصل یاد می‌گیرید

- قواعد ساخت کلید و چرا ترتیب عناصر مهم است
- الگوی Query Key Factory + هوک queryOptions برای تایپ ایمن
- invalidate با تطبیق جزئی/دقیق و predicate
- prefetchQuery، ensureQueryData، cancelQueries و بقیه ابزارها

قواعد کلیدنویسی

کلید آرایه است و از چپ به راست، از عمومی به خاص:

سلسله‌مراتب کلید

JS

```
['posts'] // همه پست‌ها
['posts', 'list'] // لیست
['posts', 'list', { page: 2, q: 'react' }] // لیست با پارامتر
['posts', 'detail', 5] // جزئیات پست ۵
['posts', 'detail', 5, 'comments'] // کامنت‌های پست ۵
```

سه قاعده:

- **متجانس و سلسله‌مراتبی:** هر بخش، پدرِ بخش بعدی است تا تطبیق جزئی (بعداً) معنا داشته باشد.
- **serializeپذیر:** فقط رشته/عدد/بولین/آرایه/شیء ساده. کلید { page: 2, q: 'react' } با { q: 'react', page: 2 } برابر است (هش پایدار با ترتیب کلیدهای مرتب‌شده).
- **متغیرها داخل کلید:** هر چیزی که خروجی را تغییر می‌دهد (پارامترها، زبان، نقش کاربر) باید در کلید باشد.

● ممنوع – کلید بی‌پارامتر برای داده پارامتری

اگر جستجوی کاربر را در State نگه دارید ولی کلید ثابت ['posts'] باشد، همه جستجوها روی یک داده نوشته می‌شوند! پارامتر باید در کلید باشد: ['posts', 'list', { q }].

الگوی Query Key Factory

رشته‌های تکراری در کل پروژه = تله غلط املایی. راه‌حل رسمی مستندات، یک آبجکت کارخانه برای هر دامنه:

```
export const postKeys = {
  all: ['posts'],
  lists: () => [...postKeys.all, 'list'],
  list: (filters) => [...postKeys.lists(), filters],
  details: () => [...postKeys.all, 'detail'],
  detail: (id) => [...postKeys.details(), id],
  comments: (id) => [...postKeys.detail(id), 'comments'],
}

// مصرف:
queryKey: postKeys.list({ page: 1, q: 'react' }) // ['posts', 'list', {...}]
queryKey: postKeys.detail(5) // ['posts', 'detail', 5]
```

جادوی این ساختار در invalidate است – از عمومی به خاص، هر سطحی را می‌توانید هدف بگیرید:

```
qc.invalidateQueries({ queryKey: postKeys.all }) // همه چیز پست‌ها
qc.invalidateQueries({ queryKey: postKeys.lists() }) // فقط لیست‌ها، نه جزئیات
qc.invalidateQueries({ queryKey: postKeys.detail(5) }) // فقط پست ۵ (+کامنت‌هایش)
```

queryOptions: کلید + queryFn در یک جا (نسخه ۵)

هوک کمکی `queryOptions` تعریف کوئری را به یک آبجکت قابل استفاده در همهٔ هوک‌ها تبدیل می‌کند و تایپ data را هم حفظ می‌کند:

```
import { queryOptions } from '@tanstack/react-query'
import { postKeys } from './keys'

export const postQueries = {
  list: (filters: { page: number; q?: string }) =>
    queryOptions({
      queryKey: postKeys.list(filters),
      queryFn: ({ signal }) => api.posts(filters, signal),
      staleTime: 30_000,
    }),
  detail: (id: number) =>
    queryOptions({
      queryKey: postKeys.detail(id),
      queryFn: ({ signal }) => api.post(id, signal),
    }),
}

// است، بدون تعریف دستی Post[] دقیقاً data مصرف – تایپ:
```

```
const { data } = useQuery(postQueries.list({ page: 1 })))
qc.invalidateQueries({ queryKey: postKeys.all })
```

invalidateQueries و فیلترها

اثر	فراخوانی
همهٔ Queryهایی که کلیدشان با <code>['posts']</code> شروع می‌شود.	<code>invalidateQueries({ queryKey: postKeys.all })</code>
فقط کلید دقیقاً برابر — بدون زیرمجموعه‌ها.	<code>invalidateQueries({ queryKey: postKeys.lists(), exact: true })</code>
فیلتر کامل با تابع — برای شرایط پیچیده.	<code>invalidateQueries({ predicate: (q) => q.queryKey[0] === 'posts' && q.state.dataUpdatedAt > hourAgo })</code>
بدون stale کردن، مستقیم <code>.refetch</code> .	<code>refetchQueries({ queryKey: postKeys.all })</code>

`invalidate` یعنی: stale کن + اگر observer فعال دارد فوراً `refetch`، وگرنه فقط برای mount بعدی آمادهٔ `refetch` باش. به یاد داشته باشید: `invalidate` هیچ چیز را حذف نمی‌کند؛ دادهٔ قدیمی تا `refetch` بعدی همان‌جاست.

ابزارهای دستوری QueryClient

متد	کاربرد واقعی
<code>getQueryData(key)</code>	خواندن کش (مثلاً snapshot در <code>onMutate</code>)
<code>setQueryData(key, updater)</code>	نوشتن کش — updaters مثل <code>setState</code> تابعی است و <code>undefined</code> را نگه می‌دارد
<code>prefetchQuery({ queryKey, queryFn })</code>	گرفتن قبل از نیاز (hover روی لینک، mount والد) — خطاها بی‌صدا رد می‌شوند
<code>ensureQueryData(key)</code>	اگر تازه در کش نیست بگیر و Promise بده — در لودرهای روتر عالی است
<code>cancelQueries(filters)</code>	لغو درخواست‌های در جریان — در <code>onMutate</code> خوش‌بینانه
<code>removeQueries(filters)</code>	حذف کامل از کش (مثلاً خروج کاربر)
<code>qc.clear()</code>	پاک‌سازی کل کش
<code>getQueryState(key) / qc.getQueryCache().findAll()</code>	دیباق و ابزارهای reporting

شد hover پیش‌واکشی هوشمند: وقتی روی لینک

JSX

```
function PostLink({ id }) {
  const qc = useQueryClient()
```

```
const prefetch = () => qc.prefetchQuery(postQueries.detail(id))

return (
  <a href={`/${id}`} onMouseEnter={prefetch} onFocus={prefetch}>
    پست {id}
  </a>
)
}
```

خروج کاربر: کش را خالی کن

JS

```
// در useAuthStore یا logout mutation
onSuccess: () => {
  queryClient.clear() // یا removeQueries({ predicate: q => q.queryKey[0] !== 'public' })
  navigate('/login')
}
```

● نکته – Structural Sharing

TanStack Query به صورت پیش فرض، پاسخ جدید را با قبلی مقایسه عمیق می کند و مرجع بخش های بدون تغییر را حفظ می کند. نتیجه: `useMemo` های شما کمتر می سوزند و React کمتر رندر می گیرد. خاموشش نکنید مگر با دلیل `(structuralSharing: false)`.

مینی الگو: invalidate هدفمند در سه سطح

شوند؟ stale ثبت کامنت – چه چیزهایی باید

JS

```
const addComment = useMutation({
  mutationFn: (vars) => api.addComment(vars.postId, vars.body),
  onSuccess: (comment, vars) => {
    // ۱) کامنت های همان پست
    qc.invalidateQueries({ queryKey: postKeys.comments(vars.postId) })
    // ۲) شمارنده کامنت در جزئیات پست
    qc.invalidateQueries({ queryKey: postKeys.detail(vars.postId) })
    // ۳) کامنت های اخیر من در داشبورد – دامنه متفاوت
    qc.invalidateQueries({ queryKey: ['my-activity'] })
  },
})
```

جمع‌بندی فصل

- کلید سلسله‌مراتبی و `serialize` پذیر بنویسید؛ پارامترها داخل کلید، نه در `State` جدا.
- `queryOptions` + `Query Key Factory` = کلید و `fetch` و تایپ در یک نقطه.
- `invalidate` تطبیق جزئی دارد؛ با `exact` یا `predicate` هدف بگیرید.
- `prefetch` برای سرعت، `ensureQueryData` برای لودرها، `remove/clear` برای خروج کاربر.
- `Structural sharing` مرجع پایدار می‌دهد؛ همان چیزی که `Zustand` هم دوست دارد.

● تمرین فصل ۱۱

برای دامنه `users` یک `Query Key Factory` کامل بنویسید (`all/lists/list/detail/posts`). بعد صفحه «پروفایل کاربر» بسازید که با `hover` روی لینک کاربر، جزئیاتش را `prefetch` می‌کند. در `logout` اپ، کل کش را پاک کنید و با `DevTools` مطمئن شوید چیزی باقی نمانده.

امکانات پیشرفته – صفحه‌بندی، Infinite و Suspense

صفحه‌بندی نرم با `placeholderData`، اسکرول بی‌نهایت با `Suspense`، `useInfiniteQuery`، `useSuspenseQuery`، کوئری‌های موازی داینامیک و اتصال به شبکه آفلاین.

در این فصل یاد می‌گیرید

- صفحه‌بندی بدون فلش با `placeholderData: keepPreviousData`
- `useInfiniteQuery` با `initialPageParam` و `getNextPageParam`
- رندر اعلامی با `useSuspenseQuery` و `ErrorBoundary`
- کوئری‌های داینامیک با `useQueries` و رفتار آفلاین

صفحه‌بندی با UI ثابت

مشکل کلاسیک صفحه‌بندی: با هر تغییر صفحه، لیست خالی و دوباره پر می‌شود؛ اسکرول می‌پرد و کاربر گیج می‌شود. راه‌حل: تا دادهٔ صفحهٔ جدید نیامده، دادهٔ صفحهٔ قبلی را «نمایشی» نگه دار:

src/components/PaginatedPosts.jsx

JSX

```
import { useState } from 'react'
import { useQuery, keepPreviousData } from '@tanstack/react-query'
import { api } from '../lib/api'
import { postKeys } from '../features/posts/keys'

export default function PaginatedPosts() {
  const [page, setPage] = useState(1)

  const { data, isPlaceholderData, isFetching } = useQuery({
    queryKey: postKeys.list({ page }),
    queryFn: ({ signal }) => api.posts(`?_page=${page}&_limit=10`, signal),
    placeholderData: keepPreviousData, // ستارهٔ ماجرا ←
    staleTime: 30_000,
  })

  return (
    <div style={{ opacity: isPlaceholderData ? 0.6 : 1 }}>
      <ul>{data?.map((p) => <li key={p.id}>{p.title}</li>)}</ul>
      <div>
        <button onClick={() => setPage((p) => p - 1)} disabled={page === 1}>قبلی</button>
        <button onClick={() => setPage((p) => p + 1)}>بعدی</button>
        {isFetching && !isPlaceholderData && <span>در حال آوردن صفحهٔ جدید</span>}
      </div>
    </div>
  )
}
```

نکته‌ها: `data` در حین `fetch` صفحه بعد، هنوز صفحه قبلی است (`isPlaceholderData: true`)؛ با `opacity` کم، حس «در حال تغییر» می‌دهیم بدون خالی‌شدن لیست. هر صفحه وقتی گرفته شد، در کش کلید خودش می‌ماند و برگشت به صفحه قبل آنی است.

useInfiniteQuery – اسکرول بی‌نهایت

برای «بارگذاری بیشتر»، ساختار داده فرق می‌کند: پاسخ شامل چند صفحه و نشانگر صفحه بعد (`CURSOR`) است. سه ورودی اجباری در نسخه ۵:

src/components/InfinitePosts.jsx

JSX

```
import { useInfiniteQuery } from '@tanstack/react-query'
import { api } from '../lib/api'
import { postKeys } from '../features/posts/keys'

export default function InfinitePosts() {
  const { data, isPending, isError, fetchNextPage, hasNextPage, isFetchingNextPage } =
    useInfiniteQuery({
      queryKey: postKeys.lists(), // برای فیلترها list({ q }) یا
      queryFn: ({ signal, pageParam }) => api.posts(`?_limit=10&_page=${pageParam}`, signal),
      initialPageParam: 1,
      getNextPageParam: (lastPage, allPages, lastPageParam) => {
        // JSONPlaceholder: ۱۰ آیتم داشت یعنی تمام
        return lastPage.length < 10 ? undefined : lastPageParam + 1
      },
      maxPages: 5, // حداکثر ۵ صفحه در کش بماند (حافظه موبایل!)
    })

  if (isPending) return <p>بارگذاری... </p>
  if (isError) return <p>!خطا</p>

  // آرایه‌ای از «پاسخ هر صفحه» است data.pages
  const posts = data.pages.flat()

  return (
    <div>
      {posts.map((p) => (
        <article key={p.id}><h4>{p.title}</h4></article>
      ))}
      <button
        onClick={() => fetchNextPage()}
        disabled={!hasNextPage || isFetchingNextPage}
      >
        {isFetchingNextPage
          ? 'در حال بارگذاری...'
          : hasNextPage
            ? 'بارگذاری بیشتر'
            : 'همه را دیدید'}
      </button>
    </div>
  )
}
```

```
)  
}
```

- `getNextPageParam`: از پاسخ آخرین صفحه بگویید صفحه بعد چیست؛ برگرداندن `undefined` یعنی تمام `( hasNextPage: false )`. اگر API شما cursor در پاسخ می‌دهد `( LastPage.nextCursor )`، همان را برگردانید.
- `data.pages` آرایه صفحات است؛ با `data.pageParams` هم نشانگرها را دارید.
- `maxPages` (جدید در v5) کش صفحات قدیمی را می‌برد تا آپ در لیست‌های خیلی بلند از حافظه نترکد.

● نکته – اتصال به IntersectionObserver

برای «اسکرول خودکار» به جای دکمه، در production با `@tanstack/react-virtual` یا یک `div sentinel` و IntersectionObserver ترکیب کنید: `if (entry.isIntersecting && hasNextPage) fetchNextPage()`. دقت کنید `fetchNextPage` را در خود کالیک observer صدا می‌زنید، نه در رندر!

useSuspenseQuery – رندر اعلامی

اگر معماری team تا React 18 + و Suspense است، `useSuspenseQuery` کامپوننت را از تمام ایف‌های وضعیت خلاص می‌کند: داده همیشه تعریف‌شده است (بدون `( data?.x )` و حالت‌های `pending/error` به مرزهای Suspense/ErrorBoundary منتقل می‌شوند:

src/components/PostDetailSuspense.jsx

JSX

```
import { Suspense } from 'react'  
import { useSuspenseQuery, ErrorBoundary } from '@tanstack/react-query'  
import { api } from '../lib/api'  
import { postKeys } from '../features/posts/keys'  
  
function PostDetail({ id }) {  
  const { data: post } = useSuspenseQuery({  
    queryKey: postKeys.detail(id),  
    queryFn: ({ signal }) => api.post(id, signal),  
  })  
  // وجود ندارد isPending همیشه هست؛ هیچ data اینجا  
  return <h1>{post.title}</h1>  
}  
  
export default function Page({ id }) {  
  return (  
    <ErrorBoundary fallbackRender={({ error }) => <p>خطا: {error.message}</p>>  
    <Suspense fallback=<p>در حال بارگذاری</p>>  
      <PostDetail id={id} />  
    </Suspense>  
    </ErrorBoundary>  
  )  
}
```

● هشدار – کلید باید زیر Suspense باشد

کامپوننتی که `useSuspenseQuery` دارد باید داخل مرز `Suspense` رندر شود، وگرنه کل مسیر تا نزدیک ترین `Suspense` بالا معلق می‌شود و کل صفحه لودینگ می‌گیرد. مرزها را دانه‌درشت نه، ولی مرتب بگذارید.

کوئری‌های موازی و داینامیک: `useQueries`

چند `useQuery` کنار هم خودبه‌خود موازی اجرا می‌شوند. اما وقتی «تعداد» کوئری‌ها داینامیک است (مثلاً جزئیات `N` کاربر انتخابی)، تعداد هوک‌ها ثابت نمی‌ماند – قانون هوک‌ها می‌شکند. راه‌حل رسمی:

کوئری با یک هوک `N`

JSX

```
import { useQueries } from '@tanstack/react-query'
import { api } from '../lib/api'
import { userKeys } from '../features/users/keys'

function UserCards({ userIds }) {
  const results = useQueries({
    queries: userIds.map((id) => ({
      queryKey: userKeys.detail(id),
      queryFn: ({ signal }) => api.user(id, signal),
      staleTime: 5 * 60_000,
    })),
  })

  return (
    <div>
      {results.map((r, i) =>
        r.isPending ? (
          <p key={userIds[i]}>...</p>
        ) : (
          <Card key={userIds[i]} user={r.data} />
        )
      )}
    </div>
  )
}
```

شبکه ناپایدار: `networkMode`

پیش‌فرض: بدون اینترنت، `Query`‌ها متوقف (`paused`) می‌شوند و با برگشتن شبکه دوباره اجرا می‌شوند – با `fetchStatus: 'paused'` می‌توانید بنر «آفلاین» نشان دهید. سه حالت:

networkMode	رفتار	کاربرد
online (پیش فرض)	بدون شبکه متوقف می شود	اکثر Query ها
offlineFirst	یک بار امتحان می کند، بعد صف می شود	داده های کش شده در SW
always	همیشه اجرا؛ شبکه مهم نیست	داده های کاملاً محلی

DevTools حرفه ای

افزونه `ReactQueryDevtools` (که در فصل ۸ نصب کردیم) بیش از نمایش لیست است:

- وضعیت هر Query (fresh/stale/paused)، زمان آخرین fetch و تعداد observer ها را می بینید.
- دکمه `invalidate/refetch/reset/copy` برای هر Query – دیباگ بدون دست زدن به کد.
- ساختار درختی کلیدها نشان می دهد factory شما دست کار می کند یا کلیدها قاطی شده اند.
- در production غیرفعال است؛ فقط dev bundle شامل آن است.

جمع بندی فصل

- صفحه بندی نرم = `placeholderData: keepPreviousData` + کلید پارامتری.
- `maxPages + (undefined یعنی تمام) Infinite = initialPageParam + getNextPageParam`.
- `Suspense = useSuspenseQuery` داخل مرز `Suspense/ErrorBoundary`؛ data همیشه هست.
- کوئری داینامیک = `useQueries` با آرایه پارامترها.
- آفلاین با `'fetchStatus': 'paused'` قابل مدیریت است؛ DevTools ابزار روزمره دیباگ کش است.

تمرین فصل ۱۲

(۱) `PaginatedPosts` را طوری ارتقا دهید که صفحه بعد را پیش واکشی کند (سرنخ: در `onSuccess` پایان `fetch` یا در `effect` با شرط `prefetchQuery`، `!isPlaceholderData` صفحه بعد). (۲) نسخه `Infinite` را با دکمه بسازید و بعد با `IntersectionObserver` خودکارش کنید. (۳) سناریوی آفلاین را تست کنید: `DevTools → Network → Offline`، بعد برگردید و رفتار `paused→refetch` را در DevTools ببینید.

پروژه عملی ۲ – اپ کامل با Zustand و TanStack Query

اپ «پست‌بلاگ» را از صفر می‌سازیم: ورود کاربر، جستجو و صفحه‌بندی، انتشار/ویرایش/حذف خوش‌بینانه – با جداسازی تمیز UI State و Server State در یک پروژه واحد.

در این فصل یاد می‌گیرید

- تقسیم کار نهایی بین Zustand و TanStack Query در یک پروژه واقعی
- اتصال (Zustand) auth به لایه API و به کش (removal در خروج)
- الگوی «فیلتر در استور → کلید در Query» برای جستجو و صفحه‌بندی
- ترکیب همه الگوها: prefetch و mutation، invalidation، optimistic

معماری: چه چیزی کجا می‌رود؟

نیاز محصول	دسته	ابزار	جای سکونت
پست‌ها، جزئیات، کامنت‌ها، کاربران	Server State	TanStack Query	کش QueryClient
توکن و پروفایل کاربر لاگین‌شده	Client (session)	Zustand + persist	useAuthStore
جستجو، صفحه فعلی، مرتب‌سازی	UI/URL State	Zustand (+ Router در واقعی)	usePostFiltersStore
باز بودن سایدبار/مودال، toastها	UI State	Zustand	useUIStore
تم روشن/تاریک	UI State	Zustand + persist	useThemeStore

● الگوی ستاره این فصل

«فیلترها در Zustand زندگی می‌کنند، ولی داده فقط از دروازه کلید Query می‌گذرد»: فیلتر جستجو State زنده UI است، اما هر ترکیب فیلتر = یک کلید کوئری متفاوت = یک ورودی کش مستقل. هیچ داده سرور به استور فیلترها وارد نمی‌شود.

ساختار پروژه

blog-app پروژه

TEXT

```
src/
├── lib/
│   └── api.js           # fetch layer + توکن از useAuthStore.getState
├── stores/
│   ├── useAuthStore.js  # persist: token, user
│   ├── usePostFiltersStore.js # q, page, sort
│   └── useUIStore.js    # theme, toast, sidebar
├── features/posts/
│   ├── keys.js         # postKeys factory
│   └── queries.js       # queryOptions های آماده
```

```

├── mutations.js           # hook های mutation
├── components/
│   ├── PostList.jsx
│   ├── PostCard.jsx
│   ├── NewPostForm.jsx
│   └── Header.jsx
└── main.jsx              # QueryClient + providers

```

(۱) لایه API + Auth Store

src/stores/useAuthStore.js

JS

```

import { create } from 'zustand'
import { persist } from 'zustand/middleware'

export const useAuthStore = create(
  persist(
    (set) => ({
      token: null,
      user: null,
      setSession: ({ token, user }) => set({ token, user }),
      clear: () => set({ token: null, user: null }),
    }),
    { name: 'auth', partialize: (s) => ({ token: s.token, user: s.user }) }
  )
)

```

src/lib/api.js

JS

```

import { useAuthStore } from '../stores/useAuthStore'
const BASE = 'https://jsonplaceholder.typicode.com'

async function request(path, { method = 'GET', body } = {}) {
  const headers = { 'Content-Type': 'application/json' }
  // خواندن توکن از استور، نه از هوک React بیرون
  const token = useAuthStore.getState().token
  if (token) headers.Authorization = `Bearer ${token}`

  const res = await fetch(BASE + path, {
    method,
    headers,
    body: body ? JSON.stringify(body) : undefined,
  })
  if (res.status === 401) {
    useAuthStore.getState().clear() // نشست منقضی - یک جا و متمرکز
    throw new Error('نشست منقضی شده؛ دوباره وارد شوید')
  }
  if (!res.ok) throw new Error(`API error ${res.status}`)
  return res.status === 204 ? null : res.json()
}

```

```
export const api = {
  posts: (params = '', signal) => request(`/posts${params}`, { signal }),
  post: (id, signal) => request(`/posts/${id}`, { signal }),
  addPost: (post) => request('/posts', { method: 'POST', body: post }),
  updatePost: (id, patch) => request(`/posts/${id}`, { method: 'PATCH', body: patch }),
  deletePost: (id) => request(`/posts/${id}`, { method: 'DELETE' }),
  login: async (email) => {
    // می‌تایم /users لاگین ندارد؛ شبیه‌سازی: کاربر را از JSONPlaceholder
    const users = await request(`/users?email=${encodeURIComponent(email)}`)
    if (!users.length) throw new Error('کاربری با این ایمیل نیست')
    return { user: users[0], token: `demo-token-${users[0].id}` }
  },
}
```

۲) فیلترها و کلیدها

src/stores/usePostFiltersStore.js

JS

```
import { create } from 'zustand'

export const usePostFiltersStore = create((set) => ({
  q: '',
  page: 1,
  setQ: (q) => set({ q, page: 1 }), // جستجوی جدید یعنی برگشت به صفحه ۱
  nextPage: () => set((s) => ({ page: s.page + 1 })),
  prevPage: () => set((s) => ({ page: Math.max(1, s.page - 1) })),
}))
```

src/features/posts/keys.js و queries.js

JS

```
import { queryOptions, keepPreviousData } from '@tanstack/react-query'
import { api } from '../../lib/api'

export const postKeys = {
  all: ['posts'],
  lists: () => [...postKeys.all, 'list'],
  list: (filters) => [...postKeys.lists(), filters],
  details: () => [...postKeys.all, 'detail'],
  detail: (id) => [...postKeys.details(), id],
}

export const postQueries = {
  list: (filters) =>
    queryOptions({
      queryKey: postKeys.list(filters),
      queryFn: ({ signal }) =>
        api.posts(`?_page=${filters.page}&_limit=9`, signal),
      placeholderData: keepPreviousData,
      staleTime: 30_000,
    })
}
```

```

    }),
    detail: (id) =>
      queryOptions({
        queryKey: postKeys.detail(id),
        queryFn: ({ signal }) => api.post(id, signal),
      }),
  },
}

```

۳ Mutations در یک فایل

src/features/posts/mutations.js

JS

```

import { useMutation, useQueryClient } from '@tanstack/react-query'
import { api } from '../../lib/api'
import { postKeys } from '../keys'
import { useUIStore } from '../../stores/useUIStore'

export function useAddPost() {
  const qc = useQueryClient()
  const toast = useUIStore((s) => s.toast)

  return useMutation({
    mutationFn: (post) => api.addPost(post),
    onMutate: async (post) => {
      await qc.cancelQueries({ queryKey: postKeys.lists() })
      const previous = qc.getQueryData(postKeys.list({ q: '', page: 1 }))
      qc.setQueryData(postKeys.list({ q: '', page: 1 }), (old) =>
        old ? [{ id: Date.now(), ...post }, ...old] : old
      )
      return { previous }
    },
    onError: (_e, _v, ctx) => {
      qc.setQueryData(postKeys.list({ q: '', page: 1 }), ctx?.previous)
      toast('انتشار ناموفق بود')
    },
    onSuccess: () => toast('ایست منتشر شد'),
    onSettled: () => qc.invalidateQueries({ queryKey: postKeys.all }),
  })
}

export function useDeletePost() {
  const qc = useQueryClient()
  return useMutation({
    mutationFn: (id) => api.deletePost(id),
    onMutate: async (id) => {
      await qc.cancelQueries({ queryKey: postKeys.lists() })
      const previous = qc.getQueryData(postKeys.list({ q: '', page: 1 }))
      qc.setQueryData(postKeys.list({ q: '', page: 1 }), (old) =>
        old?.filter((p) => p.id !== id) ?? old
      )
      return { previous }
    },
  })
}

```

```

    },
    onError: (_e, _v, ctx) =>
      qc.setQueryData(postKeys.list({ q: '', page: 1 })), ctx?.previous),
    onSettled: () => qc.invalidateQueries({ queryKey: postKeys.all }),
  })
}

```

۴) کامپوننت‌ها

src/components/PostList.jsx

JSX

```

import { useMemo } from 'react'
import { useQuery } from '@tanstack/react-query'
import { postQueries } from '../features/posts/queries'
import { usePostFiltersStore } from '../stores/usePostFiltersStore'
import { PostCard } from './PostCard'

export default function PostList() {
  const q = usePostFiltersStore((s) => s.q)
  const page = usePostFiltersStore((s) => s.page)
  const setQ = usePostFiltersStore((s) => s.setQ)
  const nextPage = usePostFiltersStore((s) => s.nextPage)
  const prevPage = usePostFiltersStore((s) => s.prevPage)

  // سبک: کلید فقط با مقدار پایدار عوض شود
  const debouncedQ = useDebounce(q, 300)

  const { data, isPending, isError, isPlaceholderData } = useQuery(
    postQueries.list({ q: debouncedQ, page })
  )

  const filtered = useMemo(
    () => (data ?? []).filter((p) => p.title.toLowerCase().includes(debouncedQ.toLowerCase())),
    [data, debouncedQ]
  )

  if (isPending) return <p>در حال بارگذاری پست‌ها</p>
  if (isError) return <p>خطا در دریافت؛ بعداً دوباره تلاش کنید</p>

  return (
    <section>
      <input
        value={q}
        onChange={(e) => setQ(e.target.value)}
        placeholder="جستجو در عنوان‌ها"
      />
      <div style={{ opacity: isPlaceholderData ? 0.6 : 1 }}>
        {filtered.map((p) => <PostCard key={p.id} post={p} />)}
      </div>
      <nav>
        <button onClick={prevPage}>قبلی</button>

```

```

        <button onClick={nextPage}>بعدی</button>
      </nav>
    </section>
  )
}

```

ورود/خروج + اتصال به کش — src/components/Header.jsx

JSX

```

import { useState } from 'react'
import { useMutation, useQueryClient } from '@tanstack/react-query'
import { api } from '../lib/api'
import { useAuthStore } from '../stores/useAuthStore'

export default function Header() {
  const user = useAuthStore((s) => s.user)
  const setSession = useAuthStore((s) => s.setSession)
  const clearAuth = useAuthStore((s) => s.clear)
  const qc = useQueryClient()
  const [email, setEmail] = useState('')

  const login = useMutation({
    mutationFn: (mail) => api.login(mail),
    onSuccess: (session) => setSession(session),
  })

  const logout = () => {
    clearAuth()
    qc.clear() // داده‌های سشن‌دار از کش هم بروند
  }

  return user ? (
    <header>
      <span>{user.name}</span>
      <button onClick={logout}>خروج</button>
    </header>
  ) : (
    <header>
      <input value={email} onChange={(e) => setEmail(e.target.value)} placeholder="email" />
      <button disabled={login.isPending} onClick={() => login.mutate(email)}>ورود</button>
    </header>
  )
}

```

جمع‌بندی معماری پروژه



- ✓ هیچ پاسخ سروری در Zustand کپی نشد.
- ✓ هیچ State UI ای در کش Query ننشست.
- ✓ توکن با `getState()` به لایه API رسید؛ خروج کاربر کش را پاک کرد.
- ✓ همه mutationها invalidation دارند؛ افزودن/حذف خوش‌بینانه با `rollback`.

● تمرین بزرگ فصل ۱۳ – پروژه شخصی

همین اپ را کامل اجرا کنید و سپس سه فیچر اضافه کنید: (۱) صفحه جزئیات پست با کامنت‌ها (کوئری وابسته + `prefetch` با `hover`). (۲) ویرایش پست با مودال (`PATCH` خوش‌بینانه). (۳) تم تاریک با `persist` که مودال‌ها هم رعایتش کنند. خروجی این تمرین را در `GitHub` بگذارید – این دقیقاً همان پورتفولیویی است که در مصاحبه نشان می‌دهید.

حرفه‌ای شو – تست، دیباگ و آمادگی استخدام

تفاوت «کد می‌زنم» و «آماده تیم هستم» همین‌جاست: تست‌های استور و هوک‌ها، دیباگ‌نوز سریع خطاهای معروف، آنتی‌پترن‌های code review و سؤالات رایج مصاحبه.

در این فصل یاد می‌گیرید

- راه‌اندازی Vitest و تست مستقیم استورهای Zustand
- تست کامپوننت‌هایی که useQuery دارند (با QueryClient ایزوله)
- دیباگ‌نوز ۶ خطای معروف و راه‌حل‌شان
- چک‌لیست code review و ۱۰ سؤال مصاحبه

راه‌اندازی تست

terminal

BASH

```
npm install -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

vite.config.js

JS

```
import { defineConfig } from 'vite'

export default defineConfig({
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: './src/test/setup.js',
  },
})
```

تست استور Zustand – بدون React!

زیباترین ویژگی استور خارج از React این است که تستش به رندر نیاز ندارد؛ مستقیم با API استور کار می‌کنید:

src/store/useCartStore.test.js

JS

```
import { describe, it, expect, beforeEach } from 'vitest'
import { useCartStore } from './useCartStore'

describe('useCartStore', () => {
  beforeEach(() => {
    // Provider تمیز قبل از هر تست - بدون رندر، بدون State
    useCartStore.setState({ items: [], couponCode: null })
  })
})
```

```

it('آیتم جدید را اضافه می‌کند', () => {
  useCartStore.getState().addItem(1)
  expect(useCartStore.getState().items).toEqual([
    { id: 1, qty: 1 }
  ])
})

it('را زیاد می‌کند qty افزودن دوباره فقط', () => {
  const { addItem } = useCartStore.getState()
  addItem(1)
  addItem(1)
  expect(useCartStore.getState().items).toEqual([
    { id: 1, qty: 2 }
  ])
})

it('تعداد صفر یعنی حذف', () => {
  useCartStore.setState({ items: [{ id: 1, qty: 1 }] })
  useCartStore.getState().setQty(1, 0)
  expect(useCartStore.getState().items).toHaveLength(0)
})

it('کد نامعتبر را رد می‌کند', () => {
  const result = useCartStore.getState().applyCoupon('NOPE')
  expect(result.ok).toBe(false)
  expect(useCartStore.getState().couponCode).toBeNull()
})
})

```

● نکته

اگر استور persist دارد، در محیط تست jsdom همان localStorage واقعی است؛ با `localStorage.clear()` در `beforeEach` یا ساخت استور با `skipHydration` در تست‌ها، از نشد بین تست‌ها جلوگیری کنید.

تست کامپوننت‌های Query – کلاینت ایزوله

قانون طلایی: در تست‌ها هرگز از `QueryClient` آپ استفاده نکنید؛ برای هر تست یک کلاینت تازه بسازید (retry خاموش تا تست‌ها سریع باشند):

src/test/utils.jsx — wrapper مشترک

JSX

```

import { render } from '@testing-library/react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'

export function renderWithClient(ui) {
  const queryClient = new QueryClient({
    defaultOptions: {
      queries: { retry: false },
      mutations: { retry: false },
    },
  })
  return render(
    <QueryClientProvider client={queryClient}>{ui}</QueryClientProvider>
  )
}

```

```
)  
}
```

src/components/PostsList.test.jsx

JSX

```
import { describe, it, expect, vi } from 'vitest'  
import { screen, waitFor } from '@testing-library/react'  
import { renderWithClient } from '../test/utils'  
import { api } from '../lib/api'  
import PostsList from './PostsList'  
  
vi.mock('../lib/api', () => ({  
  api: { posts: vi.fn() },  
}))  
  
describe('PostsList', () => {  
  it('لیست پست‌ها را نشان می‌دهد', async () => {  
    vi.mocked(api.posts).mockResolvedValue([  
      { id: 1, title: 'سلام دنیا' },  
      { id: 2, title: 'Zustand چیست' },  
    ])  
    renderWithClient(<PostsList />)  
  
    expect(screen.getByText(/بارگذاری/)).toBeInTheDocument()  
    await waitFor(() => {  
      expect(screen.getByText('سلام دنیا')).toBeInTheDocument()  
    })  
  })  
  
  it('پیام خطا را نشان می‌دهد', async () => {  
    vi.mocked(api.posts).mockRejectedValue(new Error('boom'))  
    renderWithClient(<PostsList />)  
    await waitFor(() => expect(screen.getByText(/خطا/)).toBeInTheDocument())  
  })  
})
```

دو الگوی مکمل: به جای mock کردن لایه API می‌توانید مستقیم در کش بنویسید (`qc.setQueryData(['posts'], fakeData)`) تا کامپوننت بدون شبکه تست شود؛ و برای تست خود استور Mutation، از `useMutationState` یا فراخوانی مستقیم `mutationFn` استفاده کنید.

کالبدشکافی شش خطای معروف

نشانه / پیام	علت و درمان
No QueryClient set, use QueryClientProvider	هوک Query بیرون از Provider صدا شده. Provider را بالاترین جا (main) بگذارید.
Maximum update depth exceeded Zustand	سلکتور هر بار شیء/آرایه جدید بوی گرداند (فصل ۴). سلکتور تکی یا useShallow.
همیشه رفرش می گیرد حتی تازه	staleTime پیش فرض صفر است؛ در defaultOptions تنظیم کنید.
داده بعد از mutation قدیمی می ماند	invalidation فراموش شده؛ یا کلید invalidate با کلید fetch متفاوت است – در DevTools کلیدها را مقایسه کنید.
opt update بوی گردد (آیتم حذف شده دوباره می آید)	cancelQueries قبل از setData را جا انداخته اید؛ refetch در جریان داده قدیمی را می نویسد.
سبب بعد از رفرش لحظه ای خالی دیده می شود	hydration غیرهمزمان است؛ فلگ _hasHydrated (فصل ۵).

آنتی پترن هایی که در code review رد می شوند

● فهرست ممنوعه

۱) کپی پاسخ سرور در Zustand («دو منبع حقیقت»). ۲) `useStore()` بدون سلکتور در کامپوننت های رندر-سنگین. ۳) ساخت object literal در سلکتور بدون `useShallow`. ۴) کلید رشته ای دست ساز به جای `Factory`. ۵) فراخوانی `invalidateQueries` بدون آرگومان (همه چیز را stale می کند – گاهی عمدی است، معمولاً تنبلی است). ۶) `mutateAsync` بدون `try/catch`. ۷) جهش مستقیم State بدون `fetch`. ۸) `immer` داخل `useEffect` برای داده سرور – جای آن `useQuery` است. ۹) نگاه داشتن فم های بزرگ در استور سراسری (فرم محلی بماند یا با RHF مدیریت شود). ۱۰) `persist` کردن همه استورها «به اعلام حمایت از آینده».

چک لیست code review این حوزه

- ✓ هر داده سرور فقط در کش Query است؛ Zustand فقط client state دارد؟
- ✓ کلیدها با Factory ساخته شده اند و پارامترها داخل کلیدند؟
- ✓ `staleTime` معنادار برای هر دامنه تعریف شده (نه همه صفر، نه همه بی نهایت)؟
- ✓ `mutation` ها `rollback` و `onSettled` دارند؟ خطا به کاربر نشان داده می شود؟
- ✓ خروج کاربر کش را پاک می کند؟ توکن ها از `getState` خوانده می شوند؟
- ✓ سلکتورها مقدار پایدار بوی گرداندند؟ Profiler آرام است؟
- ✓ تست استورها و کامپوننت های کلیدی پاس است؟

ده سؤال رایج مصاحبه (با پاسخ ۲۰ ثانیه‌ای)

۱. چرا Zustand و نه Context؟ سلکتور بومی و مقایسه مرجع، بدون Provider، با رندر حداقلی؛ Context برای تغییرات پرتعداد طراحی نشده است.
۲. چرا داده سرور را در Redux/Zustand نمی‌گذاریم؟ چون مالکش سرور است؛ ما فقط کش می‌خواهیم با staleTime و invalidation – دقیقاً کار Query.
۳. staleTime و gcTime چه فرقی دارند؟ اولی «چقدر داده معتبر محسوب می‌شود» و دومی «چقدر بعد از unmount در حافظه بماند».
۴. رفتار refetch خودکار چه وقت‌هاست؟ mount، window focus، reconnect – هر سه شرطی که داده stale باشد.
۵. optimistic update را شرح بده. onMutate: cancel+snapshot+set، onError: rollback، onSettled: invalidate.
۶. queryKey چرا آرایه است؟ برای هش پایدار و تطبیق سلسله‌مراتبی در invalidate.
۷. چرا set تابعی بهتر از set شی‌ای است؟ برای آپدیت‌های وابسته به State فعلی؛ از stale شدن در آپدیت‌های متوالی جلوگیری می‌کند.
۸. useShallow چه می‌کند؟ مقایسه سطحی خروجی سلکتور تا رندر اضافه نشود؛ جایگزین equalityFn حذف‌شده در v5.
۹. placeholderData و initialData فرقی‌شان؟ اولی نمایشی است و status می‌ماند pending؛ دومی داده واقعی محسوب می‌شود.
۱۰. درخواست‌های موازی تکراری چطور ددوپ می‌شوند؟ کلید یکسان → یک درخواست، همه observerها همان داده.

جمع‌بندی فصل

- تست استور بدون رندر: setState/getState مستقیم + beforeEach تمیز.
- تست Query با کلاینت ایزوله (retry: false) و mock لایه API یا setQueryData.
- شش خطای معروف را با علائمشان بشناسید؛ دیاگنوز سریع = اعتماد تیم.
- فهرست ممنوعه و چک‌لیست review را در پروژه‌تان نصب کنید (ادبیات تیمی).
- پاسخ‌های ۲۰ ثانیه‌ای مصاحبه را تمرین کنید تا روان بگویید.

● تمرین فصل ۱۴

برای پروژه فصل ۱۳ سه تست بنویسید: (۱) اکشن‌های سبد (add/double-add/qty-zero). (۲) کامپوننت PostList در دو حالت موفق و خطا. (۳) mutation حذف خوش بینانه: mock کنید deletePost یک بار reject شود و با RTL مطمئن شوید آیتم بعد از خطا دوباره در لیست است (rollback رخ داده).

چیت‌شیت، گلاساری و نقشه راه

صفحه‌ای که باید چاپش کنید: خلاصه دستورها، واژه‌نامه فارسی-انگلیسی و مسیر ادامه یادگیری بعد از این دوره.

این فصل شامل

- چیت‌شیت کامل Zustand و TanStack Query در دو بلاک کد
- گلاساری ۲۶ اصطلاح کلیدی
- نقشه راه گام‌به‌گام ادامه مسیر تا سطح Senior

چیت‌شیت Zustand

zustand-cheatsheet.js

JS

```
import { create } from 'zustand'
import { persist, devtools } from 'zustand/middleware'
import { immer } from 'zustand/middleware/immer'
import { useShallow } from 'zustand/react/shallow'
import { subscribeWithSelector } from 'zustand/middleware'

// ---- ترتیب میان‌افزار: devtools ← persist ← immer ----
const useStore = create(
  devtools(
    persist(
      immer((set, get) => ({
        // State
        count: 0,
        items: [],

        // تابع آپدیتور - sync اکشن
        inc: (by = 1) => set((s) => ({ count: s.count + by })),

        // جهش امن immer اکشن با
        addItem: (item) => set((s) => { s.items.push(item) }),

        // async اکشن
        load: async () => {
          set({ status: 'loading' })
          try {
            const data = await api()
            set({ data, status: 'success' })
          } catch (e) {
            set({ status: 'error', error: e.message })
          }
        }
      })),
    },
  ),
)
```

```

    // با get خواندن آخرین State
    double: () => set({ count: get().count * 2 }),

    // ریست
    reset: () => set({ count: 0, items: [] }),
  })),
  { name: 'key', partialize: (s) => ({ count: s.count }) }
)
)
)

// ---- مصرف در کامپوننت ----
const count = useStore((s) => s.count) // سلکتور تکی (پیش فرض)
const { a, b } = useStore(useShallow((s) => ({ a: s.a, b: s.b }))) // چند فیلد

// ---- بیرون React ----
useStore.getState().inc() // subscribe خواندن/اجرا بدون
useStore.setState({ count: 5 }) // نوشتن مستقیم
const unsub = useStore.subscribe((s) => console.log(s.count)) // گوش به همه
useStore.persist.rehydrate() // بازیابی دستی (SSR)

// ---- فرم کیرد TS: ----
// create<State>()((set) => ({ ... })))

```

چیت شیت TanStack Query

tanstack-query-cheatsheet.jsx

JSX

```

import { QueryClient, QueryClientProvider, useQuery, useMutation,
  useQueryClient, keepPreviousData, useInfiniteQuery,
  useSuspenseQuery } from '@tanstack/react-query'

// ---- راه اندازی (یک بار) ----
const qc = new QueryClient({
  defaultOptions: { queries: { staleTime: 60_000, retry: 2 } },
})
<QueryClientProvider client={qc}><App /></QueryClientProvider>

// ---- Query Key Factory ----
const postKeys = {
  all: ['posts'],
  lists: () => [...postKeys.all, 'list'],
  list: (f) => [...postKeys.lists(), f],
  detail: (id) => [...postKeys.all, 'detail', id],
}

// ---- خواندن ----
const { data, isPending, isError, error, isFetching, isPlaceholderData } =
  useQuery({
    queryKey: postKeys.list({ page }),
    queryFn: ({ signal }) => api.posts(page, signal),
  })

```

```

placeholderData: keepPreviousData, // صفحه‌بندی نرم
enabled: !!userId, // کوئری وابسته
select: (d) => d.filter(Boolean), // تبدیل
staleTime: 30_000,
})

// اسکرول بی‌نهایت
useInfiniteQuery({
  queryKey: postKeys.lists(),
  queryFn: ({ pageParam, signal }) => api.posts(pageParam, signal),
  initialPageParam: 1,
  getNextPageParam: (last, all, lastParam) => last.next ?? undefined,
})

// ---- تغییر + invalidation ----
const mutation = useMutation({
  mutationFn: (vars) => api.addPost(vars),
  onSuccess: () => qc.invalidateQueries({ queryKey: postKeys.lists() }),
})
mutation.mutate({ title })

// ---- Optimistic Update (حفظ کنید!) ----
onMutate: async (v) => {
  await qc.cancelQueries({ queryKey: postKeys.lists() })
  const prev = qc.getQueryData(postKeys.list(f))
  qc.setQueryData(postKeys.list(f), (old) => update(old, v))
  return { prev }
},
onError: (_e, _v, ctx) => qc.setQueryData(postKeys.list(f), ctx.prev),
onSettled: () => qc.invalidateQueries({ queryKey: postKeys.lists() }),

// ---- ابزارهای دستوری ----
qc.setQueryData(key, updater) // نوشتن مستقیم در کش
qc.getQueryData(key) // خواندن کش
qc.prefetchQuery({ queryKey, queryFn }) // پیش‌واکنشی (خطاها بی‌صدا)
qc.ensureQueryData({ queryKey }) // گرفتن مطمئن (لودرها)
qc.removeQueries({ queryKey }) // حذف از کش
qc.clear() // خروج کاربر

```

جدول تصمیم سریع: کجا بگذارم؟

نیاز	پاسخ کوتاه
پاسخ API را نگه دارم؟	خیر – کش Query
توکن و پروفایل کاربر؟	Zustand + persist
باز بودن مودال/تب فعال؟	useState محلی، اگر سراسری شد Zustand
فیلترها و صفحه فعلی؟	Zustand (و URL) → کلید Query
مقدار محاسبه شده از State؟	useMemo – ذخیره نکن
مقدار پرتغییر بدون رندر؟	setState/getState ترانزیشن

اصطلاح	معادل و توضیح فارسی
State	وضعیت — دادهٔ تغییرپذیری که UI از آن رندر می‌شود
Store	استور — ظرف مرکزی State و اکشن‌هایش
Action	اکشن — تابعی که State را تغییر می‌دهد
Selector	سلکتور — تابع انتخاب تکه‌ای از State برای subscribe
Re-render	رندر دوباره — اجرای دوبارهٔ کامپوننت پس از تغییر داده
Middleware	میان‌افزار — لایهٔ ارتقا‌دهندهٔ رفتار استور (persist و...)
Persist	پایدارسازی — ذخیرهٔ دائمی State در localStorage
Hydration / Rehydrate	آبرسانی — بازیابی دادهٔ ذخیره‌شده به استور پس از لود
Immutability	تغییرناپذیری — ساخت نسخهٔ جدید به جای جهش دادهٔ موجود
Shallow Compare	مقایسهٔ سطحی — برابری کلیدها/مقادیر یک سطح
Prop Drilling	حفر props — عبور اجباری داده از کامپوننت‌های میانی
Server State	وضعیت سرور — داده‌ای که مالکش سرور است؛ ما فقط کش داریم
Client State	وضعیت کلاینت — داده‌ای که مرورگر مالک حقیقی آن است
Cache	کش — انبار موقت دادهٔ سرور در کلاینت
Stale / Fresh	کهنه / تازه — آیا دادهٔ کش معتبر محسوب می‌شود؟
staleTime	مدت تازگی — مهلتی که در آن refetch خودکار لازم نیست
gcTime (cacheTime)	زمان زنده‌ماندن کش پس از بی‌observer شدن
Invalidation	ابطال — stale کردن کش پس از mutation برای refetch
Refetch	بازگیری — گرفتن دوبارهٔ داده از سرور
Prefetch	پیش‌واکشی — گرفتن داده قبل از نیاز کاربر
Optimistic Update	به‌روزرسانی خوش‌بینانه — تغییر فوری UI قبل از جواب سرور
Rollback	بازگردانی — برگرداندن UI به snapshot پیش از خطا
Query Key	کلید کوئری — شناسهٔ یکتای داده در کش
Deduplication	حذف تکرار — یک درخواست برای کلید مشترک
AbortController / signal	لغو درخواست در جریان شبکه
Suspense	مکانیزم اعلامی React برای نمایش fallback در انتظار داده

نقشه راه بعد از این دوره

- ۱ تثبیت با پروژه واقعی (۲ هفته)
پروژه فصل ۱۳ را به یک محصول کامل تبدیل کنید: auth کامل، کامنت ها، ویرایش، تست ها. این پورتفولیوی شماست.
- ۲ React Router + فرم ها (۱ هفته)
URL State را با React Router به معماری تان اضافه کنید؛ فرم های پیچیده را با Zod + React Hook Form بسازید.
- ۳ تست حرفه ای (۱ هفته)
Vitest + React Testing Library را عمیق کنید؛ Testing Trophy را بخوانید؛ CI (GitHub Actions) را برای پروژه تان فعال کنید.
- ۴ SSR و فریم ورک ها (۲-۳ هفته)
Next.js یا Remix: hydration/dehydration واقعی، Server Components، Query، و مرز دوباره Client/Server State.
- ۵ کیفیت و مقیاس (مستمر)
Performance (memoization، virtualization)، Accessibility، i18n، طراحی سیستم و code review — همان مهارت هایی که Senior را می سازند.

منابع رسمی برای ادامه

- مستندات Zustand: zustand.docs.pmnd.rs — کوتاه، دقیق و به روز.
- مستندات TanStack Query: tanstack.com/query — بخش «Guides» و «Community Resources» را از دست ندهید.
- DevTools: Redux DevTools (برای Zustand) و ReactQueryDevtools (برای Query) — روزانه استفاده کنید.
- TkDodo's Blog: «Practical React Query» — بهترین مجموعه مقالات عملی Query؛ چند الگوی این کتاب از آنجاست.

● کلام آخر

ابزارها عوض می شوند؛ معماری ذهنی نه. «مالک داده کیست؟» و «این یک کش است یا منبع حقیقت؟» — همین دو سؤال، در هر فریم ورک و کتابخانه آینده، مسیر را روشن می کند. موفق باشید؛ و سبد خریدتان همیشه hydrated!